

Introduction to EMC2

Niek Stevenson

In this introduction we will walk you through the basic steps of fitting evidence accumulation models with EMC2. The document is meant as a software introduction to the package, and is not meant to be comprehensive in terms of the theoretical background.

Before proceeding with this R script, we strongly recommend reading the EMC2 paper which provides a more comprehensive overview of the inner workings and workflow of the package.

To start with let's load in EMC2 and set a seed for reproducibility. Note that this seed does not affect the MCMC sampling, so results will vary slightly between runs.

```
rm(list=ls())  
# install.packages("EMC2")  
library(EMC2)  
set.seed(123456)
```

Data preparation

For this tutorial, we will analyze data from a seminal experiment by Forstmann et al. (2008). Their study investigated how striatum and pre-SMA facilitate decision-making under time pressure, published in the Proceedings of the National Academy of Sciences (<https://doi.org/10.1073/pnas.0805903105>).

The dataset is included in the EMC2 package as the “forstmann” dataframe. Let's create a working copy:

```
dat <- forstmann
```

Data Format

EMC2 has specific requirements for column names: - “rt” for response times (in seconds) - “R” for responses - “subjects” for participant identifiers

Note that all names are case-sensitive.

Let's examine our data:

```
head(dat)
```

	subjects	E	S	R	rt
1115	as1t	accuracy	right	right	0.4319
1116	as1t	speed	right	right	0.5015
1117	as1t	speed	left	left	0.3104
1118	as1t	accuracy	right	right	0.4809
1119	as1t	accuracy	left	left	0.3415
1120	as1t	neutral	left	left	0.3465

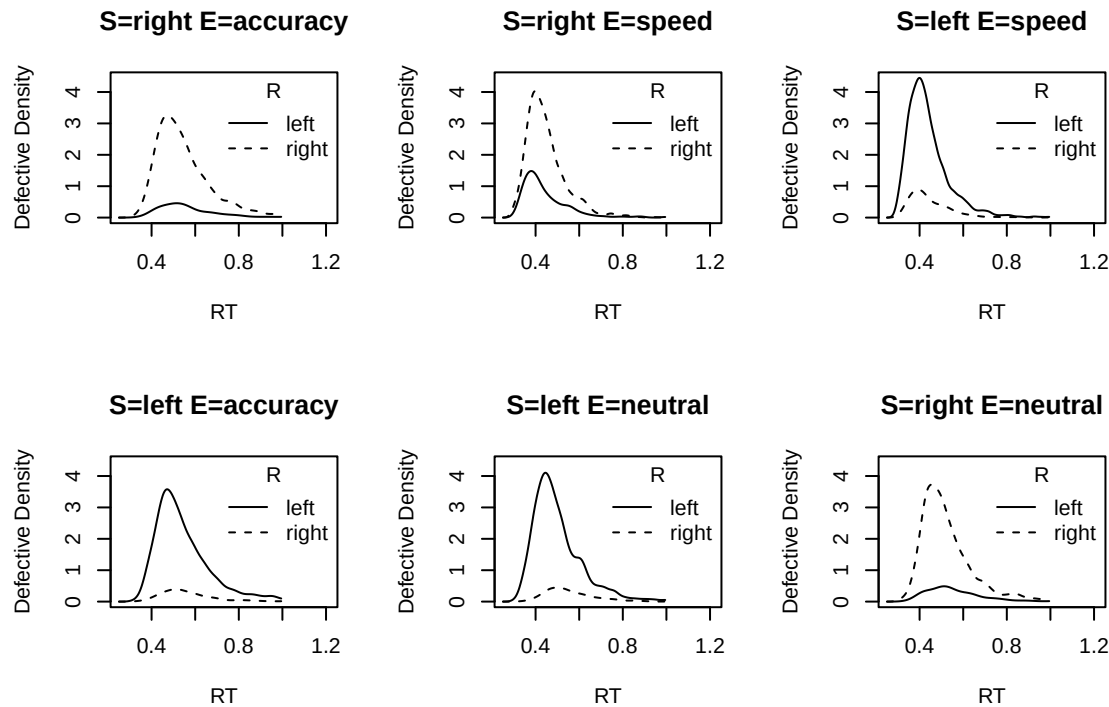
In this dataset, the response factor (R) and stimulus factor (S) share the same levels: “left” and “right”. This makes it straightforward to score correct responses by checking if S equals R. The main experimental manipulation involves cues presented before each trial, which instruct participants to emphasize either speed,

accuracy, or remain neutral. These instructions are captured in the E (emphasis) factor with three levels: “speed”, “neutral”, and “accuracy”.

The data reveals clear speed-accuracy trade-offs across conditions:

- Under speed instructions: Participants respond faster but make more errors
- Under accuracy instructions: Participants respond slower but achieve higher accuracy
- Under neutral instructions: Performance falls between the speed and accuracy conditions, but closer to the accuracy pattern

```
plot_density(dat, factors = c("S", "E"), layout = c(2,3))
```



Note: This data was previously truncated at 0.25s and 1.5s to remove outliers.

Diffusion Decision Model (DDM)

Single Subject Model

We'll begin by demonstrating how to fit a model to a single subject before moving on to hierarchical modeling. Let's extract data for only the first subject:

```
dat_single <- dat[dat$subjects == levels(dat$subjects)[1],]
dat_single$subjects <- factor(dat_single$subjects)
```

We'll now specify a DDM model using the Forstmann data. We will start with the most intuitive parameterization, where only the threshold parameter varies with emphasis condition (E).

Our model includes several key parameters:

- Drift rates (v): One each for leftward and rightward moving stimuli
- Threshold (a): Varies with the emphasis factor
- Non-decision time (t_0): Fixed across conditions

- Starting point (z): Fixed across conditions
- Drift variability (sv): Fixed across conditions

For drift rates, we use dummy coding ($v \sim 0 + S$) to estimate separate rates for left and right stimuli. The first level (left) maps to the bottom boundary and the second (right) to the top, so negative rates favor left responses and positive rates favor right responses.

Following the selective influence assumption, we allow only the threshold parameter (a) to vary with the speed-accuracy emphasis manipulation ($a \sim E$). This parameter, also known as “response caution” in DDM literature, changes based on the emphasis condition:

- Speed condition serves as the baseline level (intercept)
- Two additional parameters capture threshold differences:
 - Neutral vs. speed
 - Accuracy vs. speed

All other model parameters remain constant across conditions, with one value estimated per parameter. Since EMC2 assumes that parameters are sampled on the real line, we use the log transformation for a and t_0 to enforce that they are positive, and the probit transformation for Z , which is estimated in a proportional sense (i.e., $Z = z/a$).

To ensure model identifiability, we fix the diffusion coefficient (s) to $\log(1)$ using the constants argument. We specify $\log(1)$ since s is also estimated on the log scale as it must be positive. On the natural scale we have thus fixed s to 1. To see all the parameter transformations, see ?DDM.

Any parameters not explicitly specified in the formula receive their default values, with message telling you what they are.

Design Specification

The `design` function is arguably the most important function in EMC2. Here we specify the model type (DDM) and a data object. When the data is properly formatted, the function automatically extracts important design information that can also be specified manually. This includes:

- Response levels (`Rlevels`): The possible response options, in this case `c("left", "right")`
- Factors (`factors`): A list defining the experimental factors and their levels, such as:
 - Stimulus (S): `c("left", "right")`
 - Emphasis (E): `c("speed", "neutral", "accuracy")`

If the data contains numerical columns (besides `rt`) design will include them as a covariate. We recommend not including any unnecessary columns in your data as they can slow down model estimation.

When we run `design`, it prints the names of parameters to be sampled (8 in this case) and their corresponding design matrices. To reflect the selective influence assumption we call this “DDMaE”:

```
design_DDMaE <- design(data = dat_single,model=DDM,
                      formula =list(v~0+S,a~E, t0~1, s~1, Z~1, sv~1),
                      constants=c(s=log(1)))
```

Parameter(s) `st0`, `SZ` not specified in formula and assumed constant.

Sampled Parameters:

```
[1] "v_Sleft"      "v_Sright"     "a"            "a_Eneutral"   "a_Eaccuracy"
[6] "t0"          "Z"            "sv"
```

Design Matrices:

```
$v
```

```

      S v_Sleft v_Sright
left      1      0
right     0      1

$a
      E a a_Eneutral a_Eaccuracy
speed 1      0      0
neutral 1     1      0
accuracy 1    0      1

$t0
t0
1

$s
s
1

$Z
Z
1

$sv
sv
1

$st0
st0
1

$SZ
SZ
1

```

The print and summary functions help inspect the design at different levels of detail:

```
print(design_DDMaE)
```

For a more detailed output showing all design matrices:

```
summary(design_DDMaE)
```

```

Sampled Parameters:
[1] "v_Sleft"      "v_Sright"      "a"              "a_Eneutral"    "a_Eaccuracy"
[6] "t0"          "Z"             "sv"

Design Matrices:
$v
      S v_Sleft v_Sright
left      1      0
right     0      1

$a
      E a a_Eneutral a_Eaccuracy
speed 1      0      0

```

```

    neutral 1      1      0
    accuracy 1     0      1

$t0
t0
1

$s
s
1

$Z
Z
1

$sv
sv
1

$st0
st0
1

$SZ
SZ
1

```

Understanding how your parameterization maps to the experimental design can be challenging. The `mapped_pars` function helps build this intuition:

```
mapped_pars(design_DDMaE)
```

```

$v
S
left   : v_Sleft
right  : v_Sright

$a
E
speed   : exp(a)
neutral : exp(a + a_Eneutral)
accuracy : exp(a + a_Eaccuracy)

```

This will print a description of the transformations and mappings of the parameters back to the experimental design for all parameters that vary as a function of the experimental factors.

We can also make an illustration of the proposed design. In this case we need to construct a parameter vector (“p_vector”). To do so, we first construct a vector of example parameters, on the scales they are sampled (thus keeping the parameter transformations in mind).

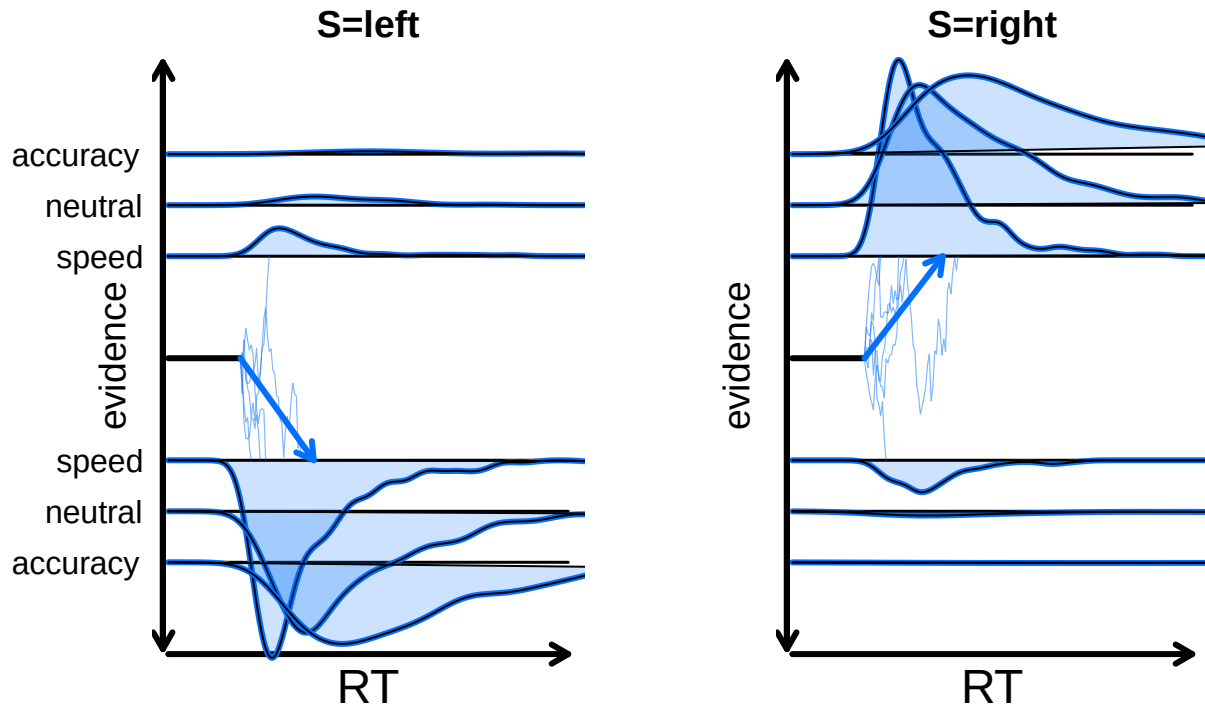
Here we’ll use some reasonable values that will later also be used to specify a prior.

```
p_vector=c(v_Sleft=-2,v_Sright=2,a=log(1),a_Eneutral=log(1.5),a_Eaccuracy=log(2),
           t0=log(.2),Z=qnorm(.5),sv=log(.5))
```

Now we can plot the design. This plot is for visual purposes, although it matches the actual proposed accumulation process of the DDM as closely as possible, some parts are altered for clarity. You might need to

adjust the plot size to fit your display:

```
plot(design_DDME, p_vector, factors = list(v = "S", a = "E"), plot_factor = "S")
```



Side note: you can also create an empty parameter vector as follows (doMap=FALSE removes the design matrices to simplify the output):

```
sampld_pars(design_DDME, doMap=FALSE)
```

v_Sleft	v_Sright	a	a_Eneutral	a_Eaccuracy	t0
0	0	0	0	0	0
Z	sv				
0	0				

Prior Specification

Single subject models assume normal priors on the parameters, requiring us to specify means and standard deviations for each of the 8 model parameters.

Note that priors are specified on the sampled scale, which requires us to keep the parameter transformations in mind when specifying the prior. As an example, we use an informed prior based on:

1. Expecting fairly accurate responding (negative rate for left, positive for right)
2. Typical DDM drift rate values around 2
3. Response caution decreasing from accuracy to neutral to speed instruction

For the prior on parameter means, we use the p_vector values defined above. Again, as an example, we found the following standard deviations to be reasonable:

```
prior_sd <- c(v_Sleft=1, v_Sright=1, a=.3, a_Eneutral=.3, a_Eaccuracy=.3,
              t0=.4, Z=1, sv=.4, SZ=1)
```

```
prior_DDME <- prior(design_DDME, pmean = p_vector, psd = prior_sd,
                    type = "single")
```

We set type is “single” because we are fitting a single subject model and we need to create a matching prior. Just as with the design, the print and summary functions can be used to inspect the prior with different levels of detail.

```
print(prior_DDME)
```

Or for a more detailed output:

```
summary(prior_DDME)
```

alpha - Subject-level prior

mean of the prior :

v_Sleft	v_Sright	a	a_Eneutral	a_Eaccuracy	t0
-2.0000000	2.0000000	0.0000000	0.4054651	0.6931472	-1.6094379
Z	sv				
0.0000000	-0.6931472				

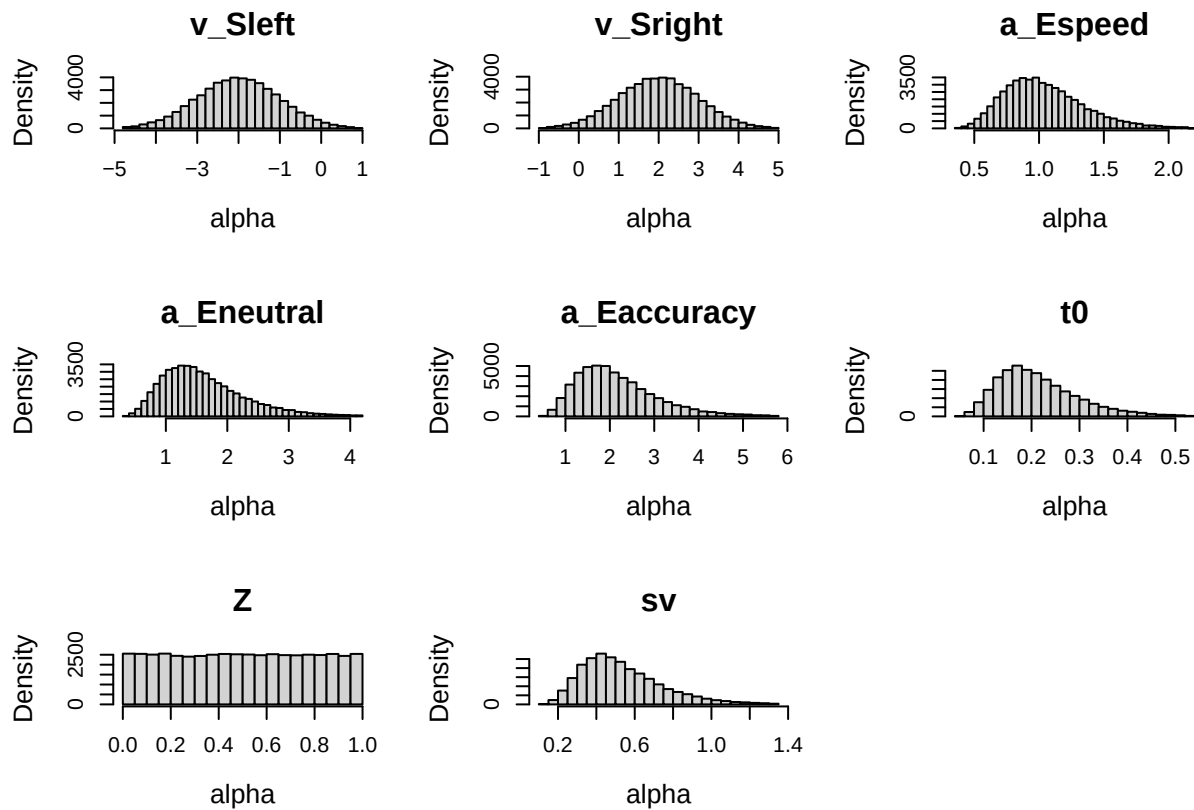
variance of the prior :

	v_Sleft	v_Sright	a	a_Eneutral	a_Eaccuracy	t0	Z	sv
v_Sleft	1	0	0.00	0.00	0.00	0.00	0	0.00
v_Sright	0	1	0.00	0.00	0.00	0.00	0	0.00
a	0	0	0.09	0.00	0.00	0.00	0	0.00
a_Eneutral	0	0	0.00	0.09	0.00	0.00	0	0.00
a_Eaccuracy	0	0	0.00	0.00	0.09	0.00	0	0.00
t0	0	0	0.00	0.00	0.00	0.16	0	0.00
Z	0	0	0.00	0.00	0.00	0.00	1	0.00
sv	0	0	0.00	0.00	0.00	0.00	0	0.16

To check if our prior is reasonable, we can visualize it using the plot function. The default plotting behavior maps parameters back to their natural scales.

Visualizing this mapping is important because parameter transformations can have counterintuitive effects. For example, when using the probit transformation, a normal prior with mean=0 and SD=1 actually creates a uniform prior on the mapped scale.

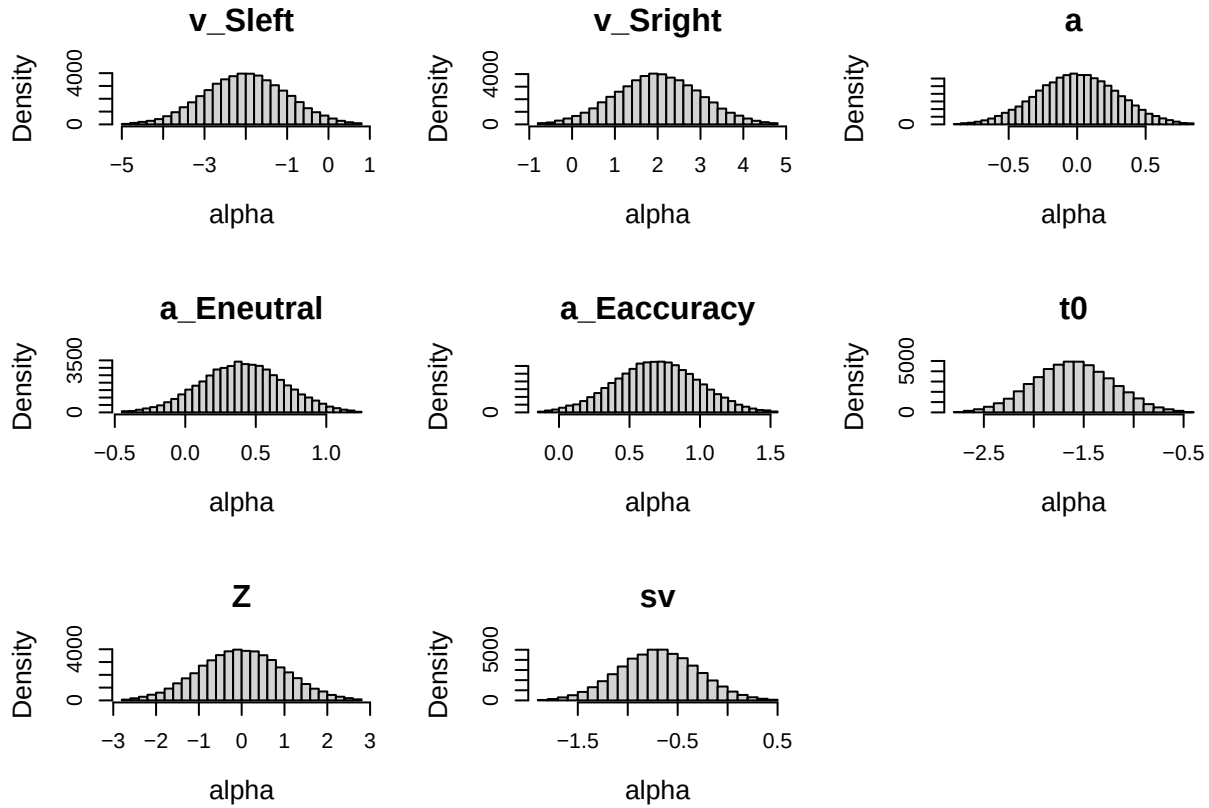
```
plot(prior_DDME)
```



Note that there is slight un-evenness in the plots as they are obtained by simulation, which is why these functions can take some time to run.

We can also plot the prior on the unbounded scale, using the `map=FALSE` option. This will show the normal distributions on the sampled scale.

```
plot(prior_DDME, map = FALSE)
```

Similarly, we can also inspect quantiles of the prior using the `credint` function.

```
credint(prior_DDmaE)
```

```
$alpha
      2.5%   50%  97.5%
v_Left    -3.951 -2.019 -0.048
v_Sright    0.111  1.962  3.818
a_Espeed    0.563  0.986  1.773
a_Eneutral  0.665  1.462  3.442
a_Eaccuracy 0.870  1.998  4.550
t0          0.094  0.201  0.457
Z           0.026  0.512  0.981
sv          0.225  0.509  1.114
```

Lastly we can again inspect the proposed mapping using the `mapped_pars` function, which now uses the prior means as inputs:

```
mapped_pars(prior_DDmaE)
```

```
      E      S  v   a  sv  t0 st0 s   Z SZ    z  sz
1  speed left -2 1.0 0.5 0.2  0 1 0.5  0 0.50  0
2  neutral left -2 1.5 0.5 0.2  0 1 0.5  0 0.75  0
3 accuracy left -2 2.0 0.5 0.2  0 1 0.5  0 1.00  0
4  speed right  2 1.0 0.5 0.2  0 1 0.5  0 0.50  0
5  neutral right  2 1.5 0.5 0.2  0 1 0.5  0 0.75  0
6 accuracy right  2 2.0 0.5 0.2  0 1 0.5  0 1.00  0
```

This returns a matrix with columns for every parameter type. For example, the threshold parameter ‘a’ is mapped as:

- Speed condition: $\exp(\log(1)) = 1$
- Neutral condition: $\exp(\log(1) + \log(1.5)) = 1.5$
- Accuracy condition: $\exp(\log(1) + \log(2)) = 2$

The output also includes additional parameters that aren't directly specified but can be useful for interpretation. Note, for example, the difference between relative bias (Z) (which is proportional to the response boundary a) and absolute bias (z).

Our prior looks reasonable, so we can now fit the model.

Model Estimation

First, we combine the data, design and prior in an emc object. By default `make_emc` creates an object with three chains. It assumes that response time is recorded with a resolution of 0.02 seconds (assuming a 50Hz refresh rate of a monitor and hence in most experimental setups the maximum resolution for recording response times to visual stimuli).

This specification compresses the data to speed up sampling by:

1. Keeping only rows with unique combinations of parameters, responses and rt values
2. Computing log-likelihood calculations only for these unique values
3. Replicating and summing the log-likelihoods appropriately

Since we're only fitting a single subject model, we again specify the single type.

```
DDMaE <- make_emc(dat_single, design_DDMaE, prior = prior_DDMaE, type = "single")
```

Processing data set 1

Likelihood speedup factor: 3.5 (232 unique trials)

Next we run the parameter estimation (sampling). By default 3 x 1000 samples will be obtained after burn in and adaptation (but possibly more if this is not enough to satisfy default convergence criteria). By default EMC2 parallelizes across the (in this case 3) chains. This can be turned off by setting `cores_for_chains = 1`.

The `fileName` argument saves intermediate progress (as an object called "emc") to a file, which is useful for longer runs in case they need to be restarted. Sampling can be slow so you just load the results instead of running the next chunk (this is more so for the hierarchical modeling we will look at next).

```
DDMaE <- fit(DDMaE, fileName = 'tmp.RData')
save(DDMaE, file = "DDMaE.RData")
```

Notice how sampling sped up towards the end of the sampling phase? That's because the number of particles is adaptively scaled down as the proposals get more concentrated around the posterior mode.

The print function gives a quick overview of the estimated model:

```
print(DDMaE)
```

Iterations:

	preburn	burn	adapt	sample
[1,]	0	0	0	1000
[2,]	0	0	0	1000
[3,]	0	0	0	1000

Subjects:

```
[1] "as1t"
```

Parameters:

```
[1] "v_Sleft"      "v_Sright"     "a"            "a_Eneutral"   "a_Eaccuracy"
```

```
[6] "t0"          "Z"          "sv"
```

The check function serves two purposes: 1. Verifies that sampling has converged 2. Quantifies sampling efficiency through the “effective” number of samples (ESS)

ESS represents the number of samples after discounting auto-correlation. When performing posterior inference (e.g., estimating 95% credible intervals), it’s important to have sufficient effective samples for stable estimates.

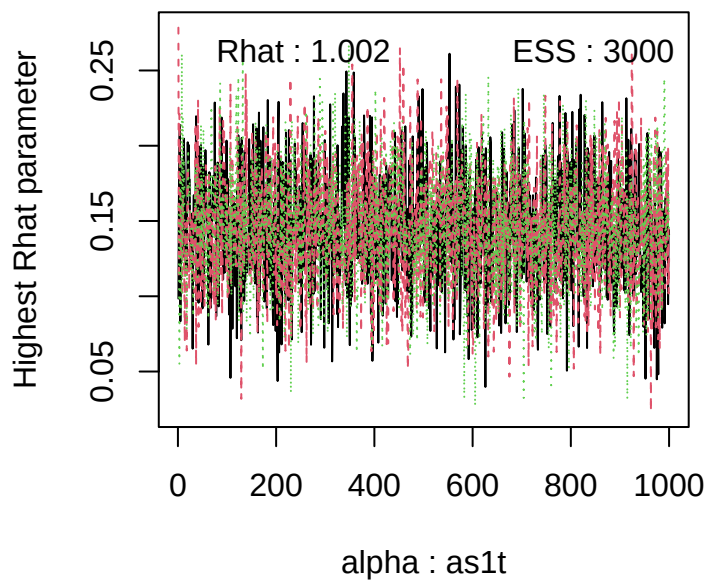
```
check(DDMaE)
```

Iterations:

	preburn	burn	adapt	sample
[1,]	0	0	0	1000
[2,]	0	0	0	1000
[3,]	0	0	0	1000

	as1t	v_Sleft	v_Sright	a	a_Eneutral	a_Eaccuracy	t0	Z	sv
Rhat		1.001	1	1	1.002	1.001	1	1	1.002
ESS		2951.000	3521	3186	3000.000	3000.000	3000	3003	2490.000

Trace of a_Eneutral



The check function also displays plots of the worst converging chains for each parameter type (in this case only the **alpha** type, since we fit a single subject model), as measured by the Gelman-Rubin statistic (Rhat/psrf). This makes it easy to visually identify any convergence issues in the sampling.

For a more comprehensive analysis of the MCMC chains, we can use the summary function. This provides detailed statistics for each parameter, including the Rhat, ESS, and key quantiles of the posterior distribution.

```
summary(DDMaE)
```

	alpha	as1t	2.5%	50%	97.5%	Rhat	ESS
v_Sleft			-1.372	-1.126	-0.879	1.001	2951
v_Sright			1.191	1.446	1.691	1.000	3521
a			-0.089	-0.029	0.030	1.000	3186

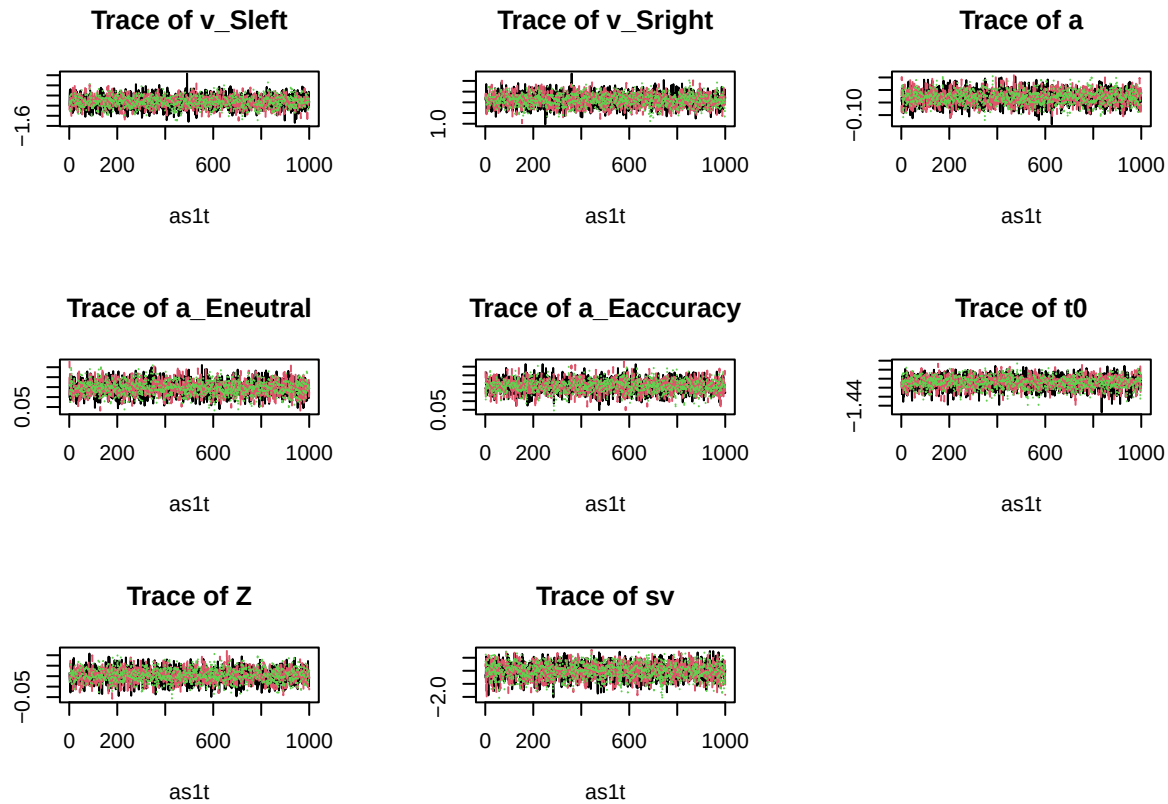
```

a_Eneutral  0.070  0.146  0.221  1.002  3000
a_Eaccuracy 0.106  0.190  0.266  1.001  3000
t0          -1.429 -1.414 -1.401  1.000  3000
Z           -0.014  0.053  0.123  1.000  3003
sv          -1.654 -0.976 -0.410  1.002  2490

```

We can also look at the full set of MCMC chains which show well converged the chains are.

```
plot(DDMaE)
```

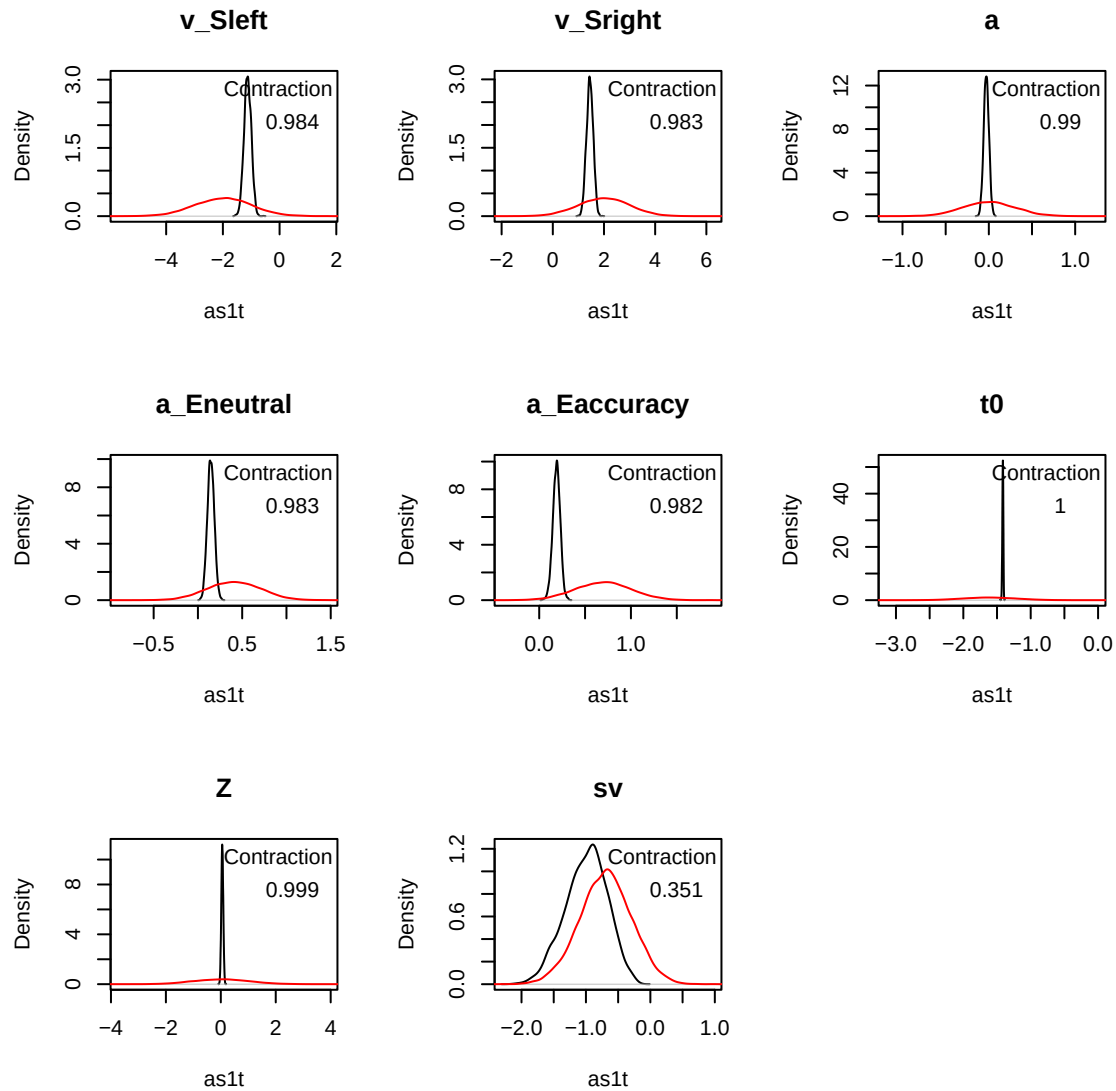


Model Inference

The `plot_pars` function can be used to plot posterior densities for each parameter (black) superimposed on priors (red) to check how much the priors have been “updated” by the data. Ideally, we want priors to be “dominated” by the data, meaning the red line is relatively flat while the black line is sharply peaked.

The dominance is quantified by a contraction statistic: one minus the ratio of posterior to prior variance. Values close to 1 indicate that the prior has been effectively washed out by the data.

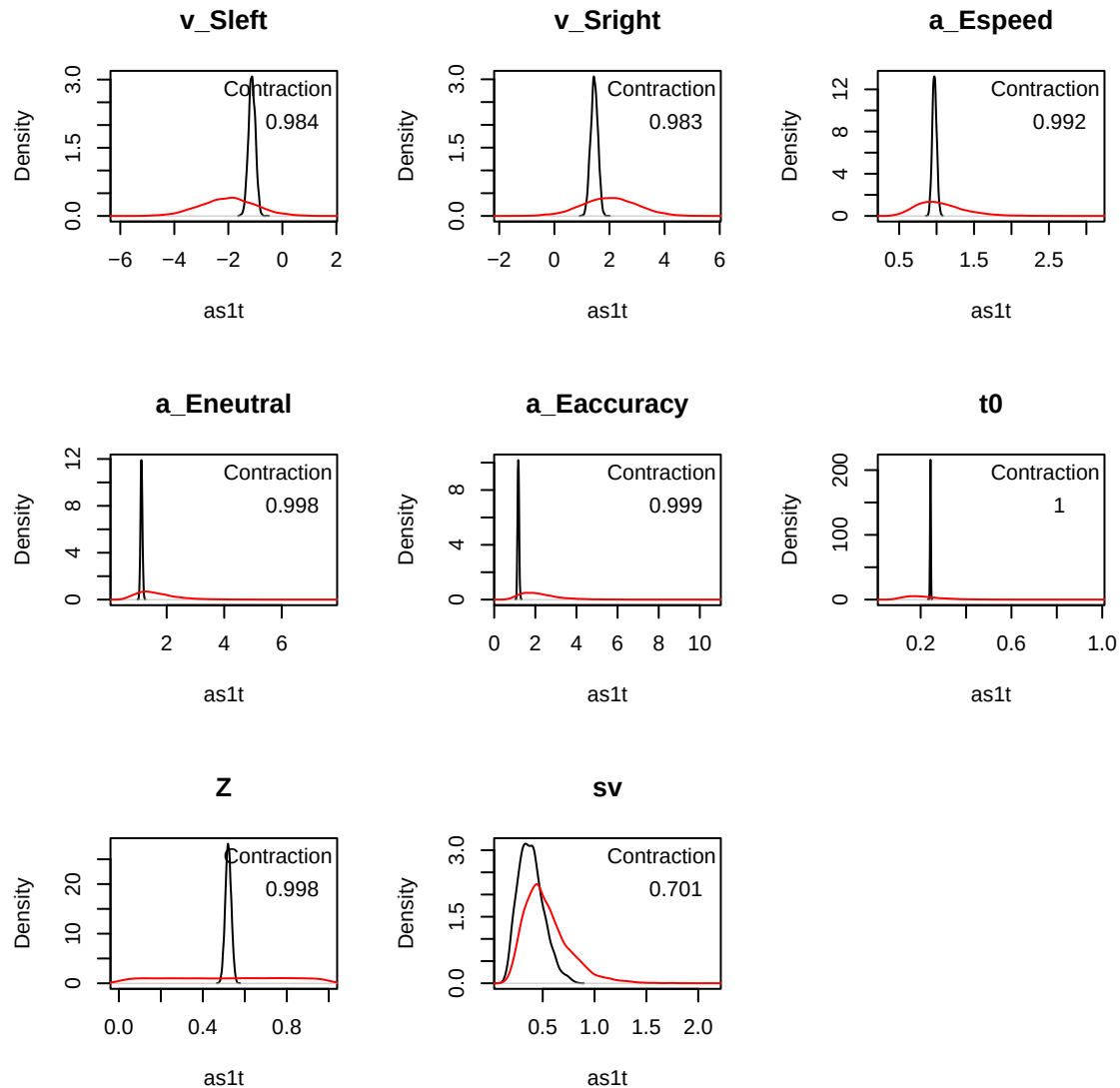
```
plot_pars(DDMaE)
```



Note that the prior on **sv** wasn't nearly as washed out as the other parameters, indicating that the data didn't provide much information about this parameter.

We can also plot the parameter posteriors as they are mapped back to the original design:

```
plot_pars(DDMaE, map=TRUE)
```



We can assess how well the model fits the data visually using posterior predictives, which are calculated by:

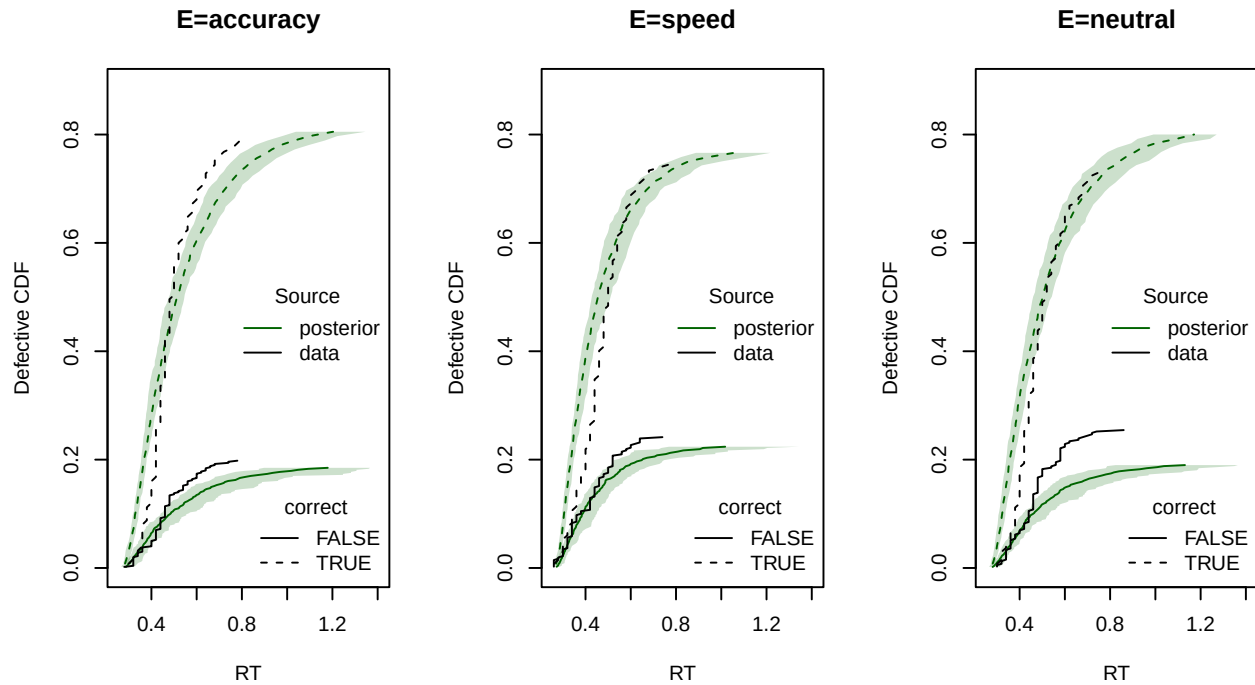
1. Sampling parameter sets from the posterior
2. Using these parameters to simulate new data

This can be slow, so using parallel processing through the `n_cores` argument is a good idea.

```
pp_DDmaE <- predict(DDmaE, n_cores = 12)
```

We can quantify misfit using defective cumulative distribution function plots. Here we add a function to the data that creates a new column "correct":

```
acc_fun <- function(data) return(data$S == data$R)
plot_cdf(DDmaE, pp_DDmaE, factors = "E", functions = c(correct = acc_fun),
  layout = c(1,3), defective_factor = "correct",
  legendpos = c("bottomright", "right"))
```

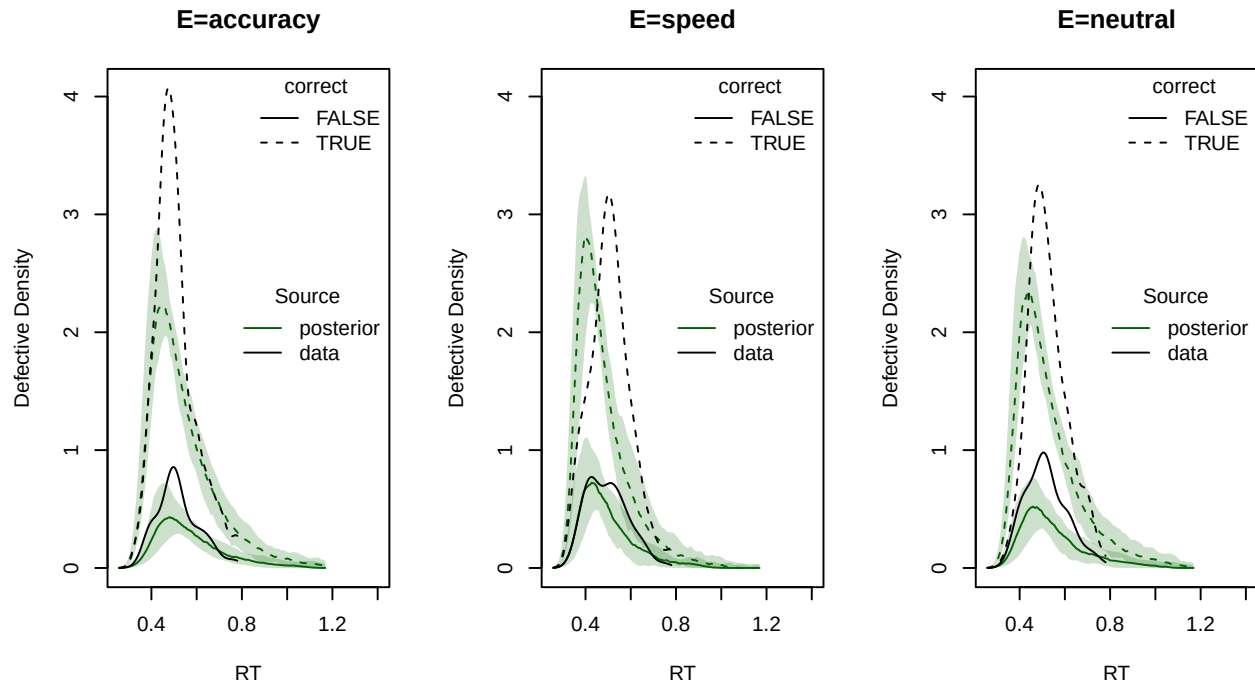


In these plots:

- Green line: Median of predicted data
- Light green shading: 95% credible intervals of the posterior predictives
- Black line: Observed data
- X-axis: Response time in seconds
- Y-axis: Cumulative defective density (probability of response by that time)

Similarly, we can plot the defective density of the posterior predictives over the real data:

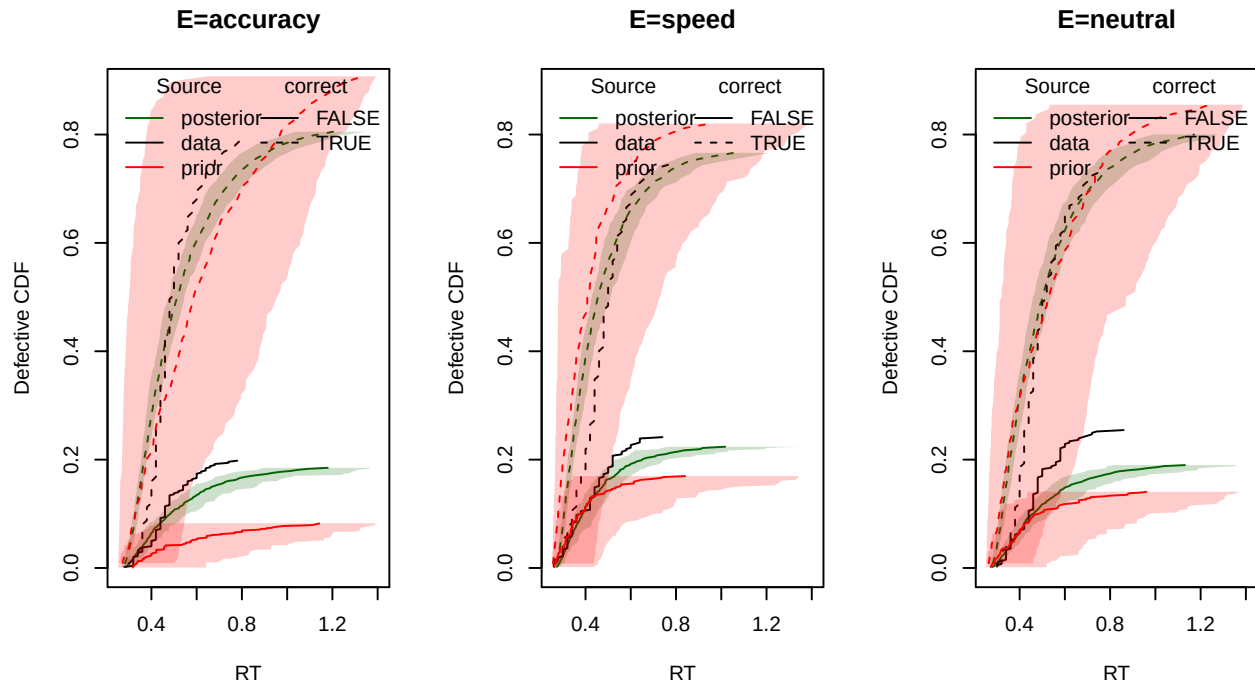
```
plot_density(DDMaE, pp_DDMaE, factors = "E", functions = c(correct = acc_fun),
  layout = c(1,3), defective_factor = "correct",
  legendpos = c("topright", "right"))
```



Note that our model does not seem to fit the data well. Most notably, response times are overestimated and number of error responses is underestimated

We can also include prior predictives in `plot_cdf` or `plot_density`. To do this: we set `to_plot = c('real', 'posterior', 'prior')`. Alternatively, we can create and save prior predictives using `predict(prior_DDmaE)`. By default, plot limits are based on real data and posterior predictives. Include prior predictives in limits using the `use_lim` parameter

```
plot_cdf(DDmaE, pp_DDmaE, factors = "E", functions = c(correct = acc_fun),
  layout = c(1,3), defective_factor = "correct",
  legendpos = c("topright", "topleft"),
  to_plot = c('real', 'posterior', 'prior'))
```

```
# or: priorp_DDMaE <- predict(prior_DDMaE)
# plot_cdf(DDMaE, pp_DDMaE, priorp_DDMaE, ...)
```

Obviously our prior predictions are quite broad.

Hierarchical Model

We will now move on to hierarchical estimation of the DDM. Given the poor fit (albeit on just one subject), let's break the conventional selective influence assumption and allow response emphasis (E) to also influence drift rates.

Instead of using different drift rates for left and right stimuli, for each E level we will reparameterize the drift rates into:

1. A drift bias: The average rate across stimuli
2. A drift rate difference: The difference between stimuli

This can be achieved by specifying a contrast matrix for the S factor that takes the difference between right and left stimuli:

```
Smat <- matrix(c(-1,1), nrow = 2,dimnames=list(NULL,"dif"))
```

To see how this works, we alter the previous design with the formula $v \sim S$, supplying Smat in the contrasts argument:

```
new_design_DDMaE <- design(data = dat,
  contrasts = list(S = Smat),
  formula=list(v~S,a~E, t0~1, s~1, Z~1, sv~1),
  constants=c(s=log(1)),
  model=DDM)
```

Parameter(s) st0, SZ not specified in formula and assumed constant.

```
Sampled Parameters:
[1] "v"          "v_Sdif"      "a"           "a_Eneutral" "a_Eaccuracy"
```

```
[6] "t0"          "Z"          "sv"
```

Design Matrices:

```
$v
      S v v_Sdif
left 1      -1
right 1       1

$a
      E a a_Eneutral a_Eaccuracy
speed 1          0          0
neutral 1         1          0
accuracy 1         0          1
```

```
$t0
t0
1
```

```
$s
s
1
```

```
$Z
Z
1
```

```
$sv
sv
1
```

```
$st0
st0
1
```

```
$SZ
SZ
1
```

Let's examine the parameter mapping:

```
mapped_pars(new_design_DDmaE)
```

```
$v
      S
left   : v - v_Sdif
right  : v + v_Sdif

$a
      E
speed   : exp(a)
neutral : exp(a + a_Eneutral)
accuracy : exp(a + a_Eaccuracy)
```

The intercept (v) represents the drift bias - the average of rates for left and right stimuli:

- Negative values indicate bias towards left-moving stimuli (lower boundary)

- Positive values indicate bias towards right-moving stimuli (upper boundary)
- Zero represents no bias

The difference parameter (v_Sdif) represents the conventional “drift rate”:

- Positive values indicate stronger accumulation towards correct responses
- Higher values suggest better discrimination ability between stimuli

Design Specification

Now we can use this contrast in a model where drift rate (v_Sdif) varies with E. We expect that accuracy emphasis causes participants to pay more attention to the stimuli and better discriminate left and right stimuli.

Since response emphasis is unlikely to affect rate bias, we’ll fix bias effects for speed vs. neutral ($v_Eneutral$) and speed vs. accuracy ($v_Eaccuracy$) to zero in the constants argument. This means v will index the same rate bias across all E levels. Since both a and v are influenced by E, we call this the DDMavE design:

```
design_DDMavE <- design(data = dat,model=DDM,
  contrasts = list(S = Smat),
  formula=list(v~S*E,a~E, t0~1, s~1, Z~1, sv~1),
  constants=c(s=log(1), v_Eneutral = 0, v_Eaccuracy = 0))
```

Parameter(s) $st0$, SZ not specified in formula and assumed constant.

Sampled Parameters:

[1] "v"	"v_Sdif"	"v_Sdif:Eneutral"	"v_Sdif:Eaccuracy"
[5] "a"	"a_Eneutral"	"a_Eaccuracy"	"t0"
[9] "Z"	"sv"		

Design Matrices:

```
$v
  S      E v v_Sdif v_Eneutral v_Eaccuracy v_Sdif:Eneutral
left speed 1  -1      0      0      0
left neutral 1  -1      1      0     -1
left accuracy 1  -1      0      1      0
right speed 1   1      0      0      0
right neutral 1   1      1      0      1
right accuracy 1   1      0      1      0
v_Sdif:Eaccuracy
0
0
-1
0
0
1
```

```
$a
  E a a_Eneutral a_Eaccuracy
speed 1      0      0
neutral 1     1      0
accuracy 1     0      1
```

```
$t0
t0
```

```

1

$s
s
1

$Z
Z
1

$sv
sv
1

$st0
st0
1

$$SZ
SZ
1

```

Let's have a look at how our parameters map to the cells of the data:

```
mapped_pars(design_DDMavE)
```

```

$v
  S      E
left  speed      : v - v_Sdif
left  neutral    : v - v_Sdif + v_Eneutral - v_Sdif:Eneutral
left  accuracy   : v - v_Sdif + v_Eaccuracy - v_Sdif:Eaccuracy
right speed      : v + v_Sdif
right neutral    : v + v_Sdif + v_Eneutral + v_Sdif:Eneutral
right accuracy   : v + v_Sdif + v_Eaccuracy + v_Sdif:Eaccuracy

$a
  E
speed      : exp(a)
neutral    : exp(a + a_Eneutral)
accuracy   : exp(a + a_Eaccuracy)

```

Prior Specification

For the hierarchical model we specify:

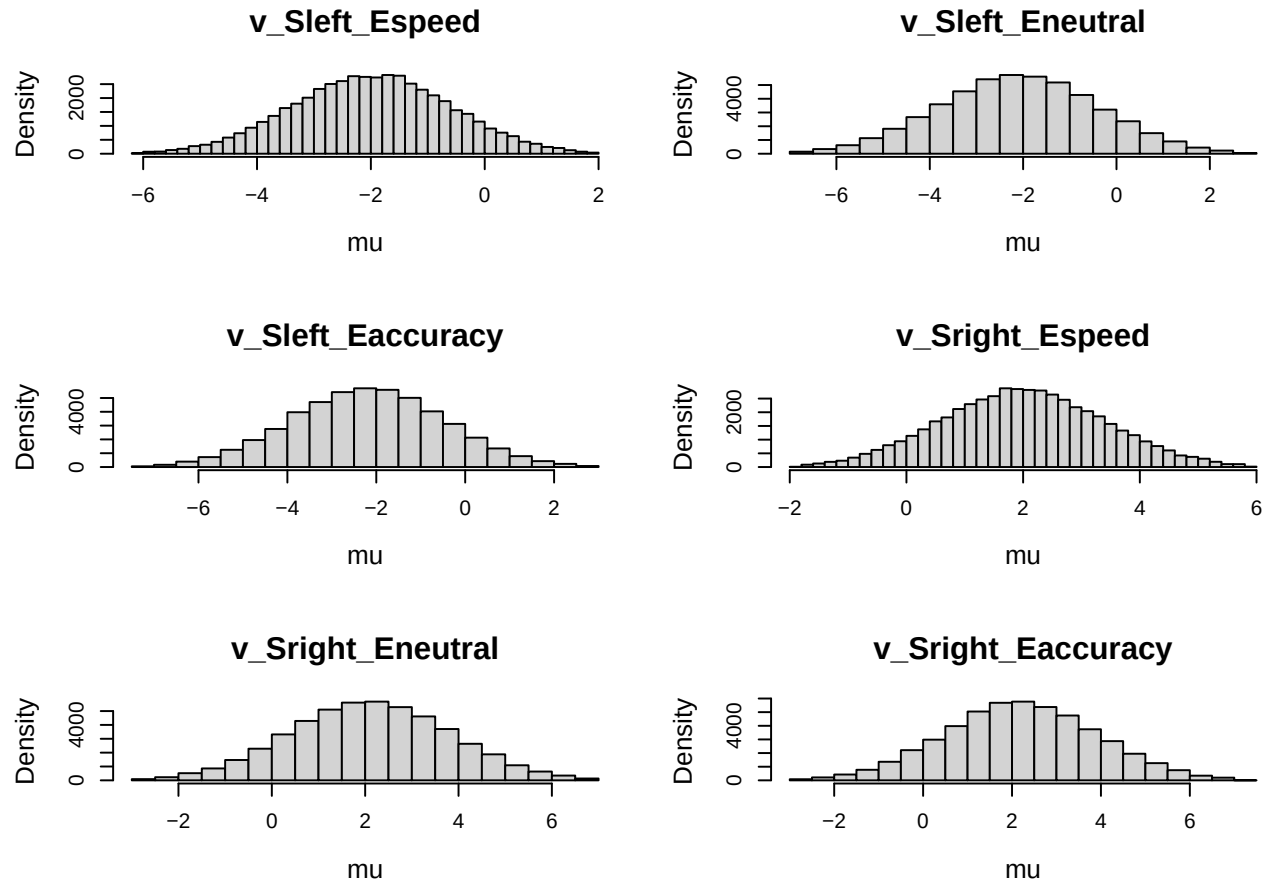
1. A prior on the group-level mean
2. A prior on the variance-covariance matrix

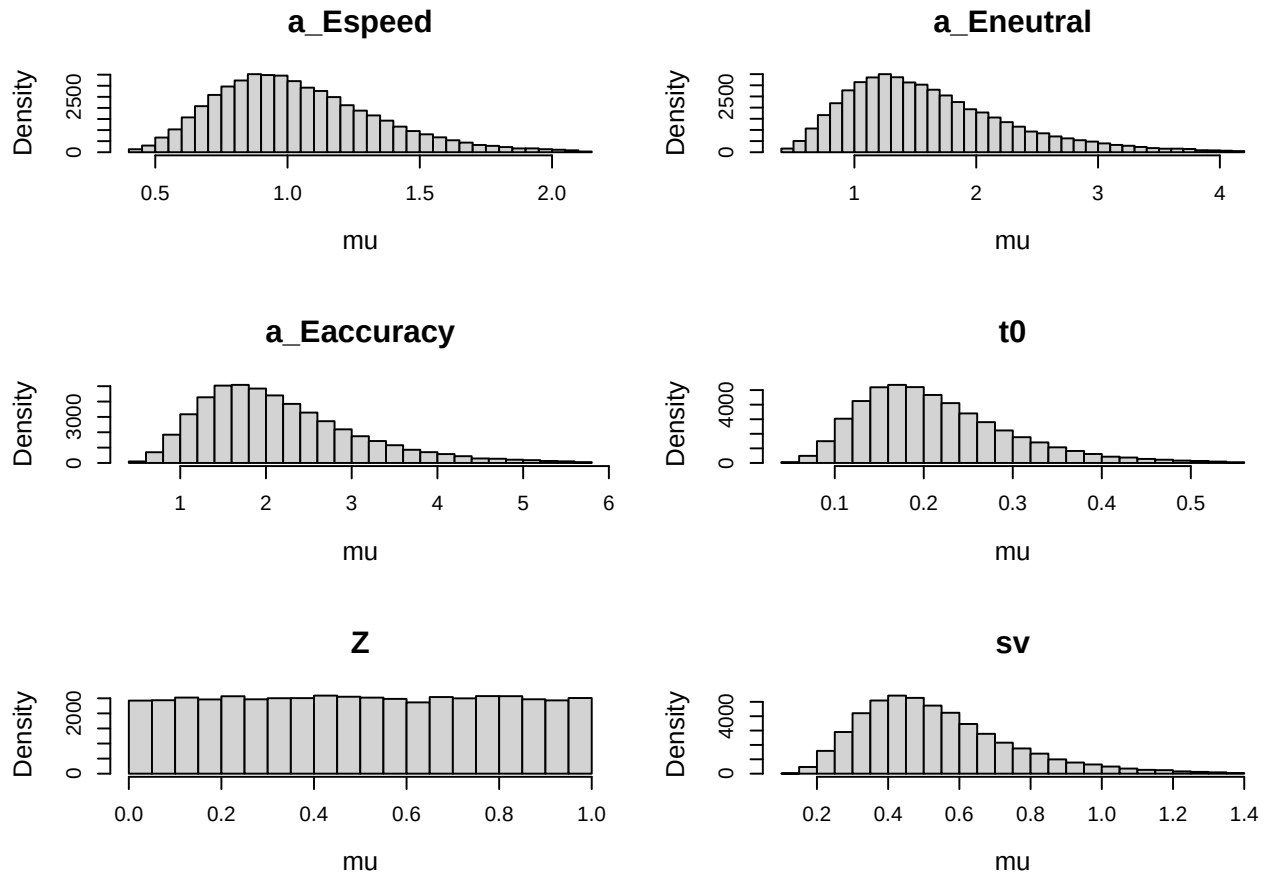
For the group-level mean, we need to specify prior means and standard deviations for each parameter. We can reuse values from the previous prior for the single subject model by supplying the previous prior to the update argument. This means we only need to specify values for the 4 new parameters.

For the intercept and drift rates, we'll use values that yield the same implied effect on left and right rates as in the previous model. We set `type = standard` to indicate this is a prior for a hierarchical model with full variance-covariance matrix:

```
prior_DDMaxE <- prior(design_DDMaxE, update = prior_DDMaxE,
  mu_mean=c(v=0,v_Sdif=2,'v_Sdif:Eneutral'=0.1,
    'v_Sdif:Eaccuracy'=0.2),
  mu_sd=c(v=1,v_Sdif=1,'v_Sdif:Eneutral'=1,
    'v_Sdif:Eaccuracy'=1), type = "standard")
```

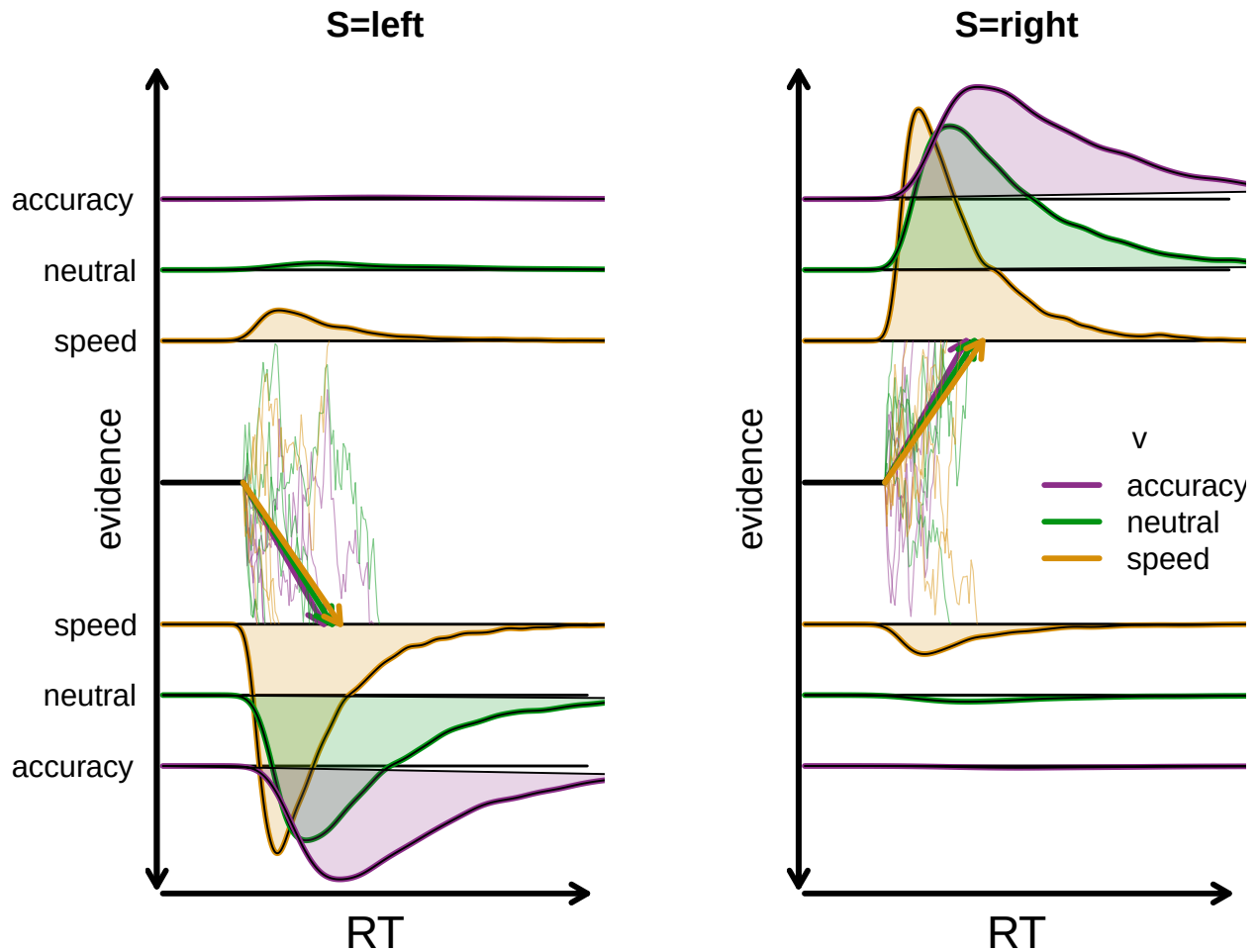
Let's check our choices look reasonable
`plot(prior_DDMaxE)`





We can also give the prior to the `plot_design` function. This gives analogous results to `plot(design)`, but uses the prior mean as the parameter input:

```
plot_design(prior_DDMavE, factors = list(v = c("S", "E"), a = "E"),
            plot_factor = "S")
```



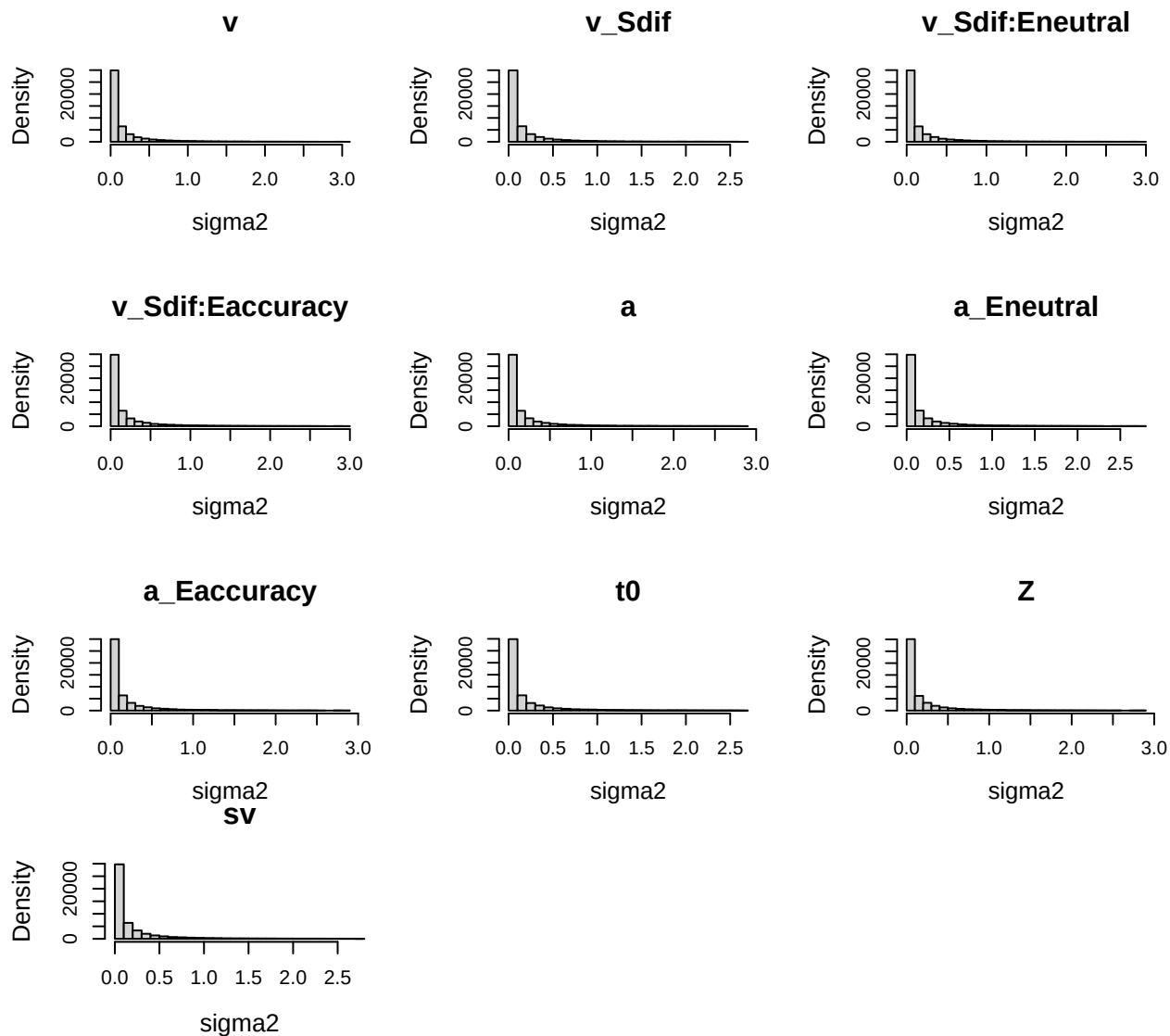
We can also specify priors on the group-level covariance matrix. This prior is best considered in two parts:

1. Variances: Default priors are vague but favor smaller values
2. Correlations: Default priors are uniform on the -1 to 1 interval

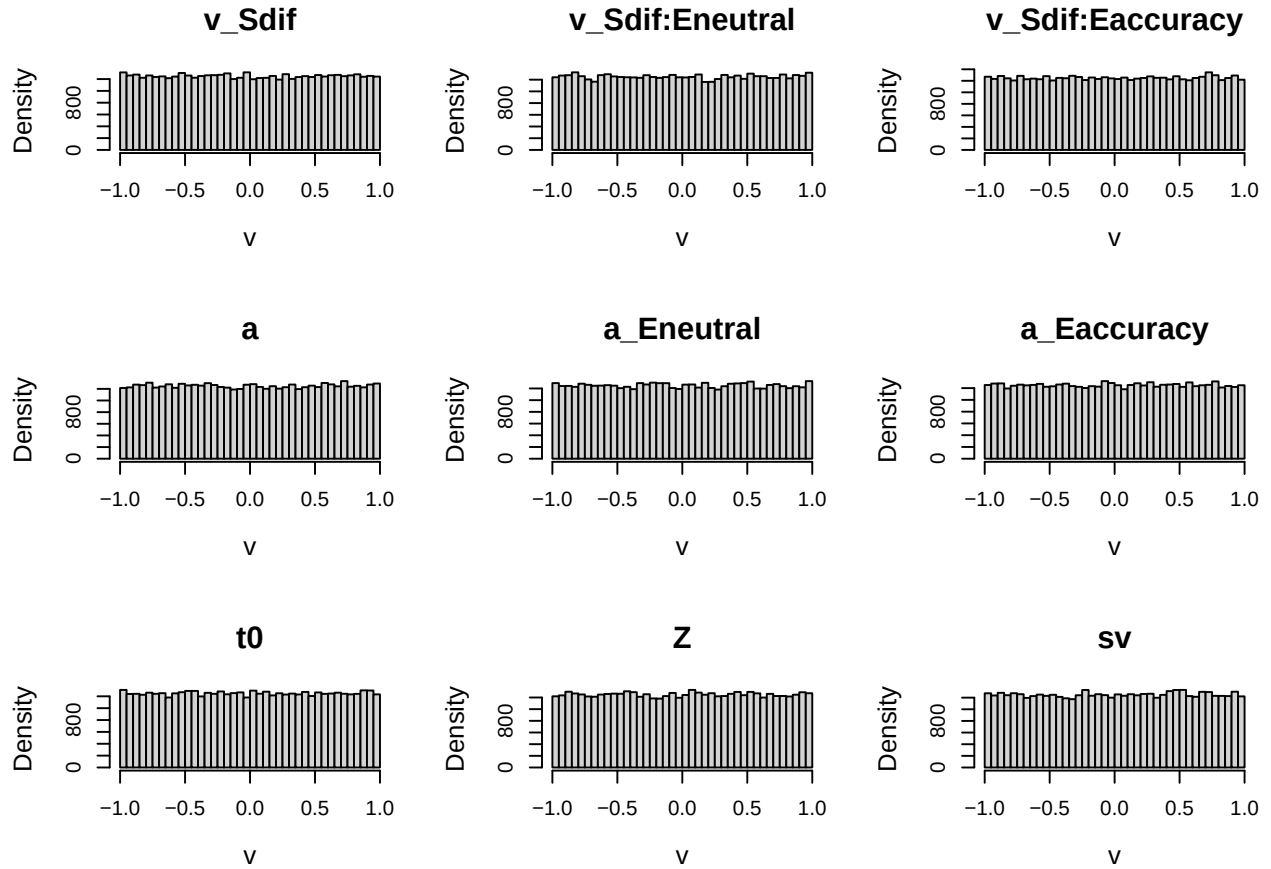
Note that here the default is mapped=FALSE and there is no mapped=TRUE option, since the variances cannot be easily mapped to the natural scale.

We only plot the prior on the correlations between other parameters and the v parameter to reduce the output, but the prior is the same across all correlations.

```
plot(prior_DDME, selection = "sigma2", layout = c(3,3))
```



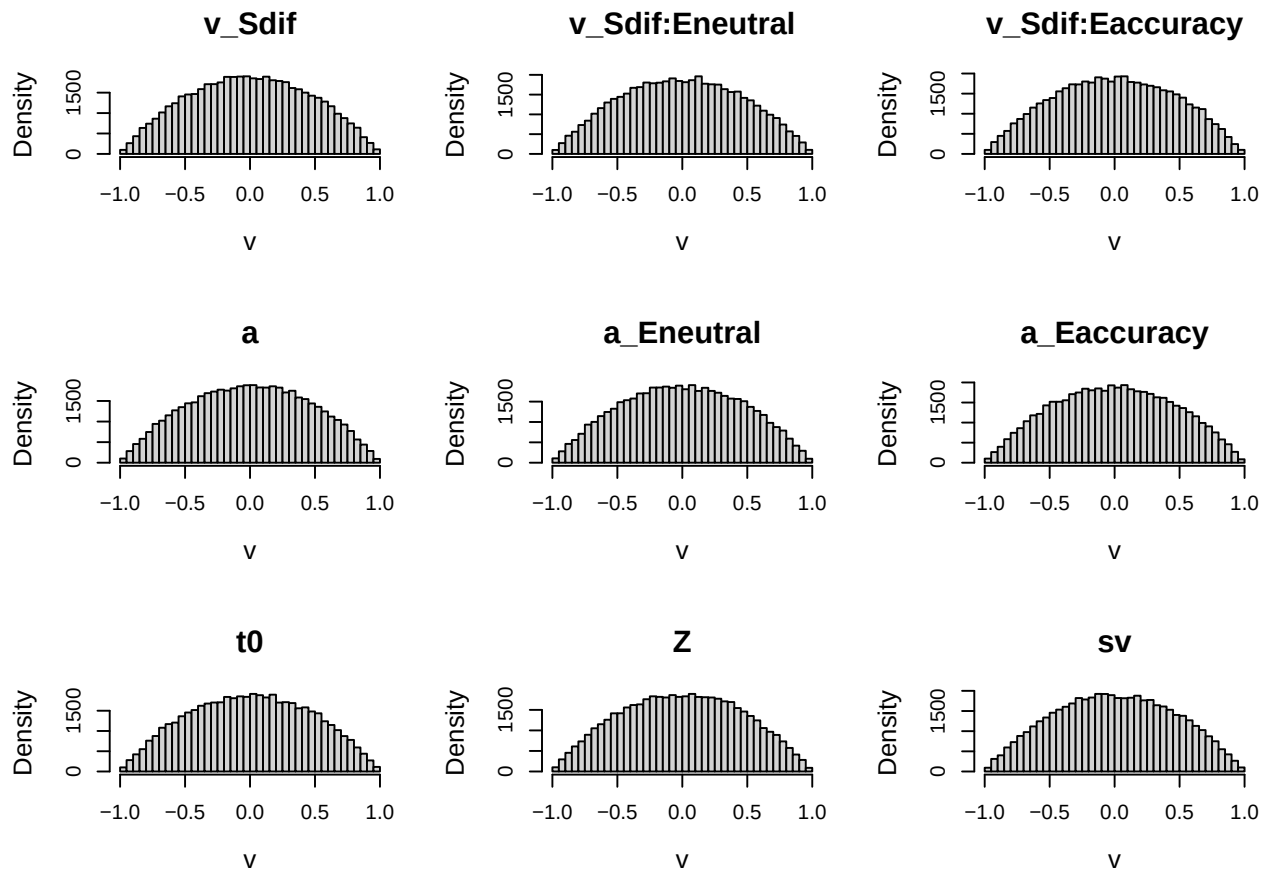
```
plot(prior_DDMavE, selection = "correlation", use_par = "v")
```

The variance-covariance prior settings can be adjusted using two hyperparameters: - Parameter A controls the variance width - Parameter v controls the correlations

For example, we can make the correlation prior more peaked at zero by increasing v from its default of 2 to 4. However, this flexibility is limited as the same prior applies to all correlations:

```
plot(prior(design_DDMavE,v=4,update=prior_DDMavE),
     selection = "correlation", use_par = "v")
```



To get more details on the prior structure you can use the summary function, which will now include information on the prior on the group-level mean and group-level variance:

```
summary(prior_DDmavE)
```

```
mu - Group-level mean
```

```
mean of the group-level mean prior :
```

	v	v_Sdif	v_Sdif:Eneutral	v_Sdif:Eaccuracy
	0.0000000	2.0000000	0.1000000	0.2000000
	a	a_Eneutral	a_Eaccuracy	t0
	0.0000000	0.4054651	0.6931472	-1.6094379
	Z	sv		
	0.0000000	-0.6931472		

```
variance of the group-level mean prior :
```

	v	v_Sdif	v_Sdif:Eneutral	v_Sdif:Eaccuracy	a	a_Eneutral
v	1	0	0	0	0.00	0.00
v_Sdif	0	1	0	0	0.00	0.00
v_Sdif:Eneutral	0	0	1	0	0.00	0.00
v_Sdif:Eaccuracy	0	0	0	1	0.00	0.00
a	0	0	0	0	1	0.09
a_Eneutral	0	0	0	0	0	1
a_Eaccuracy	0	0	0	0	0	0
t0	0	0	0	0	0	0
Z	0	0	0	0	0	0
sv	0	0	0	0	0	0

```

v                0.00 0.00 0 0.00
v_Sdif           0.00 0.00 0 0.00
v_Sdif:Eneutral  0.00 0.00 0 0.00
v_Sdif:Eaccuracy 0.00 0.00 0 0.00
a                0.00 0.00 0 0.00
a_Eneutral       0.00 0.00 0 0.00
a_Eaccuracy      0.09 0.00 0 0.00
t0               0.00 0.16 0 0.00
Z                0.00 0.00 1 0.00
sv               0.00 0.00 0 0.16

```

Sigma - Group-level covariance matrix

degrees of freedom on the group-level (co-)variance prior, 2 leads to uniform correlations. Single value [1] 2

scale on the group-level variance prior, larger values lead to larger variances :

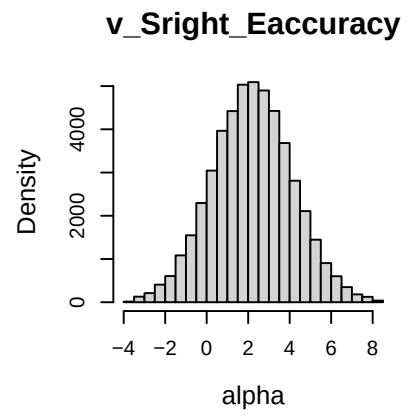
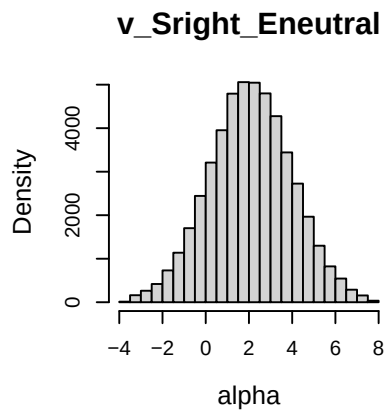
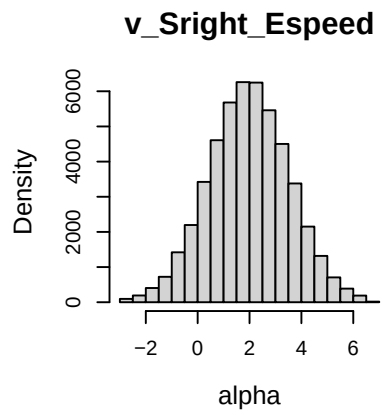
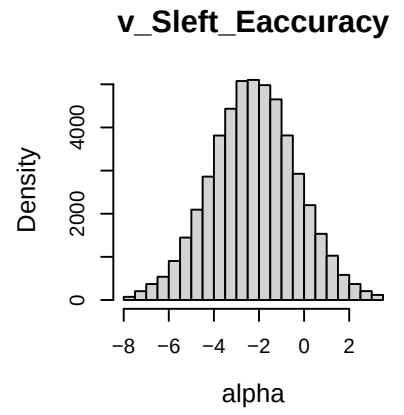
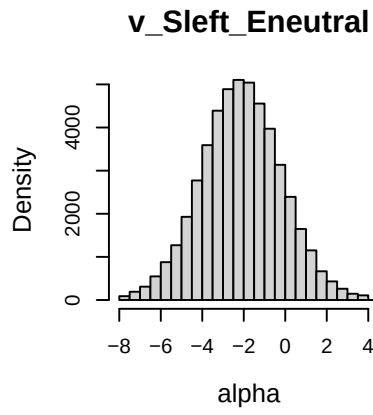
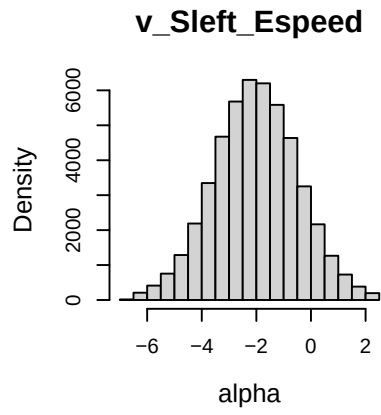
```

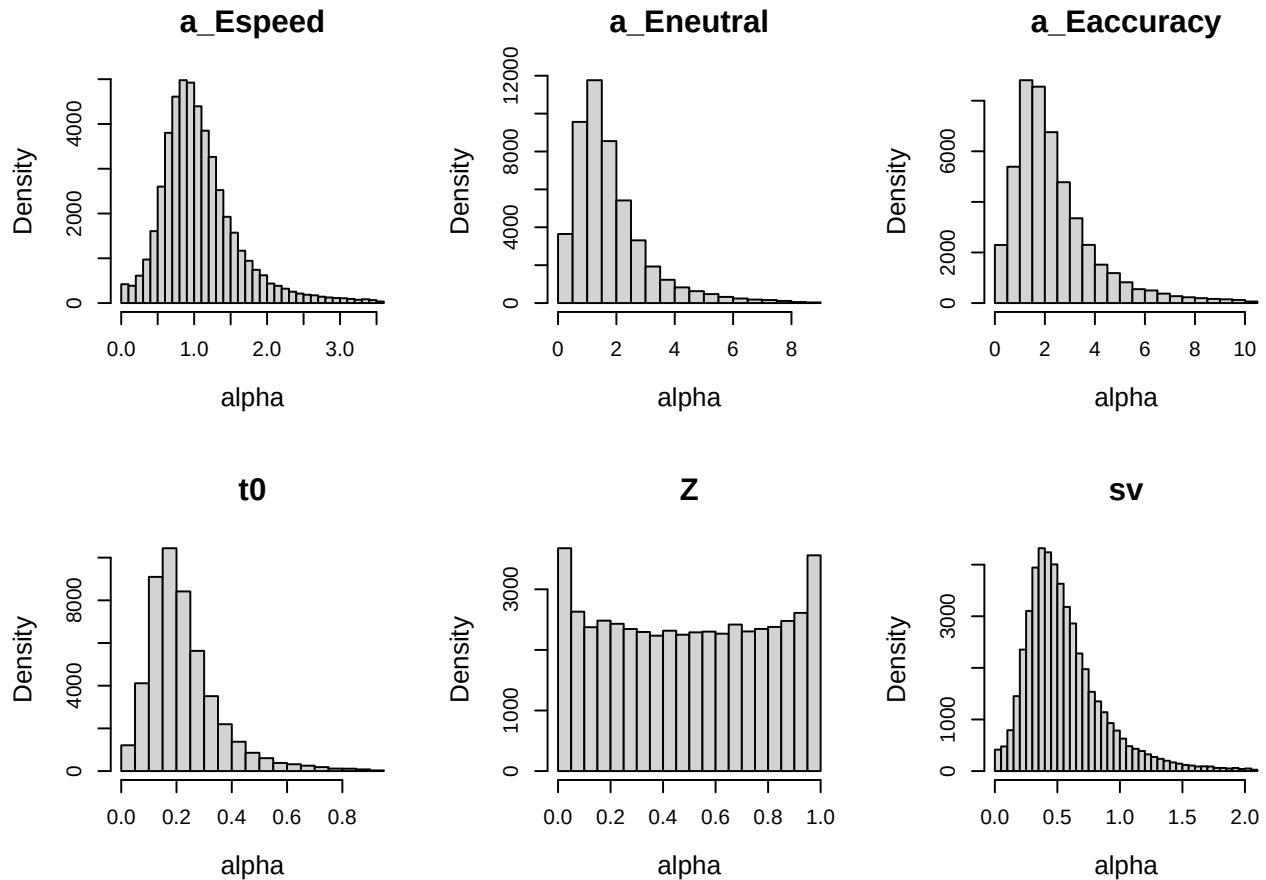
      v      v_Sdif v_Sdif:Eneutral v_Sdif:Eaccuracy
0.3      0.3      0.3      0.3
      a      a_Eneutral      a_Eaccuracy      t0
0.3      0.3      0.3      0.3
      Z      sv
0.3      0.3

```

We can also plot the “implied” priors at the individual level by sampling from the group-level distribution. This prior depends on the group level distribution, so when the group level distribution is updated during sampling, the implied prior becomes less vague. This creates the “shrinkage” effect that makes hierarchical modeling valuable:

```
plot(prior_DDMavE, selection = "alpha", layout = c(2,3))
```





Model Estimation

We now fit the model using two levels of parallelization:

1. Across chains: Default behavior (3 chains)
2. Within chain: Set `cores_per_chain = 4`

This results in 12 total cores (3 chains $\times$ 4 cores per chain). Note that within-chain parallelization is not available on Windows systems.

```
DDMavE <- make_emc(dat, design = design_DDMavE, prior = prior_DDMavE)
DDMavE <- fit(DDMavE, cores_per_chain = 4, fileName = "tmp.RData")
save(DDMavE, file = "DDMavE.RData")
```

After sampling we again check for convergence.

```
check(DDMavE)
```

Iterations:

	preburn	burn	adapt	sample
[1,]	0	0	0	1000
[2,]	0	0	0	1000
[3,]	0	0	0	1000

	mu	v	v_Sdif	v_Sdif:Eneutral	v_Sdif:Eaccuracy	a	a_Eneutral
Rhat		1.003	1.002	1.003	1.003	1	1.001
ESS		2052.000	2296.000	1466.000	1075.000	2330	2642.000

	a_Eaccuracy	t0	Z	sv
Rhat	1.001	0.999	1.001	1.006
ESS	2742.000	2388.000	1892.000	380.000

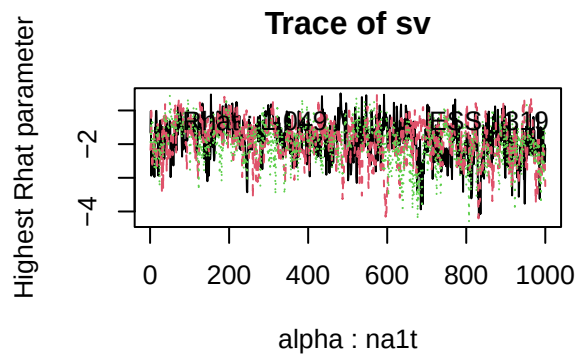
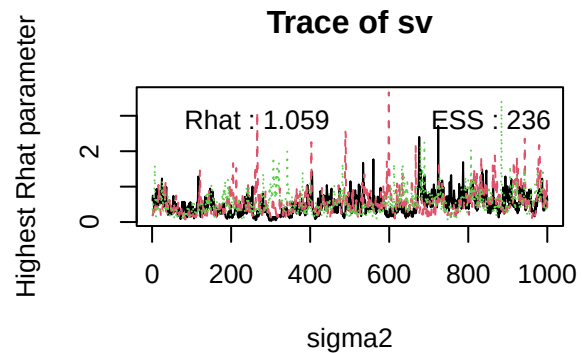
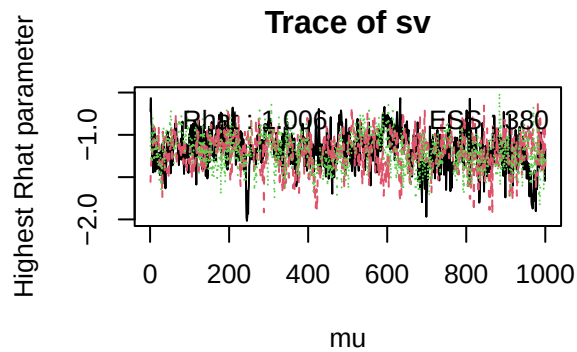
	v	v_Sdif	v_Sdif:Eneutral	v_Sdif:Eaccuracy	a	a_Eneutral
Rhat	1.001	1.002	1.008	1.008	1.003	1.002
ESS	1336.000	2122.000	938.000	781.000	1520.000	2247.000

	a_Eaccuracy	t0	Z	sv
Rhat	1.002	1.001	1.007	1.059
ESS	2194.000	1346.000	768.000	236.000

alpha highest Rhat : na1t

	v	v_Sdif	v_Sdif:Eneutral	v_Sdif:Eaccuracy	a	a_Eneutral
Rhat	1.003	1.003	1.004	1.004	1	1.001
ESS	1512.000	1180.000	905.000	791.000	1886	2360.000

	a_Eaccuracy	t0	Z	sv
Rhat	1.003	1.001	1.003	1.049
ESS	1975.000	2328.000	1418.000	319.000



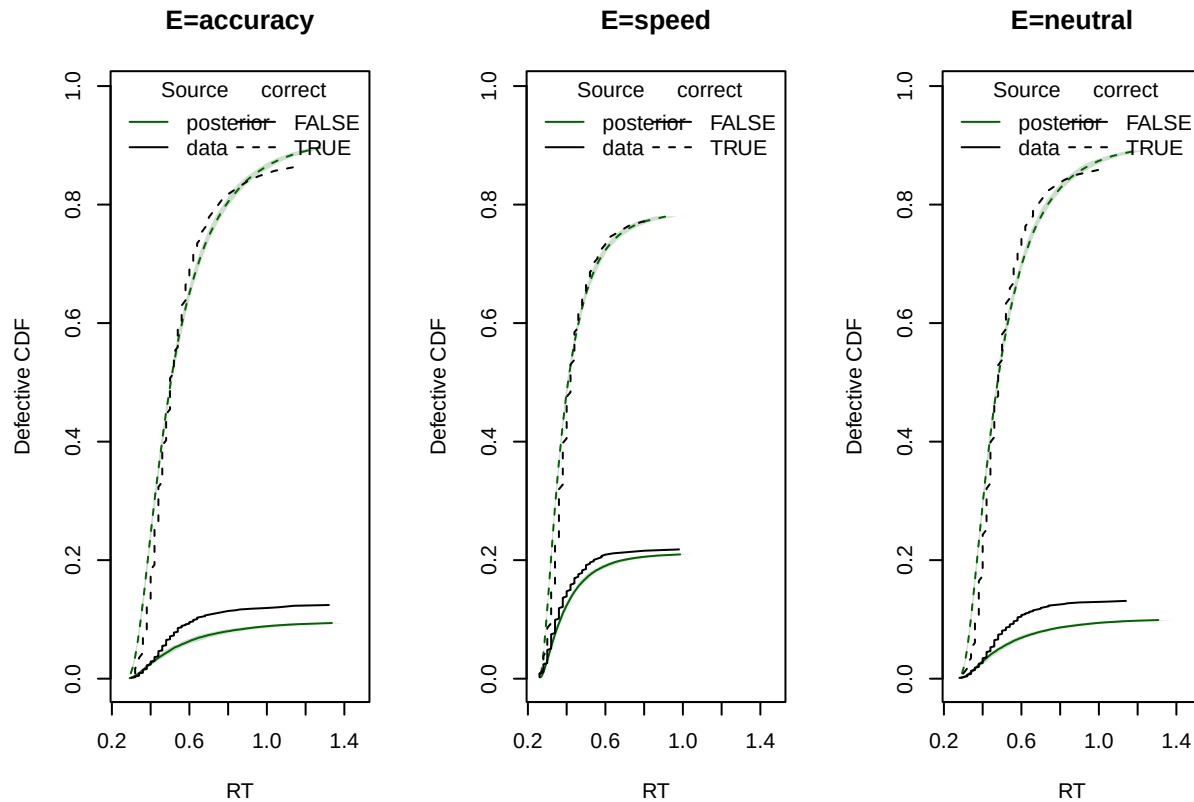
Model Inference

Model Fit Let's first inspect posterior predictives. We set `n_cores = 12` to speed up computations:

```
pp_DDMavE <- predict(DDMavE, n_cores = 12)
```

The model seems to fit reasonably well, but there's still an underestimation of the proportion of error responses, most notably in the neutral condition.

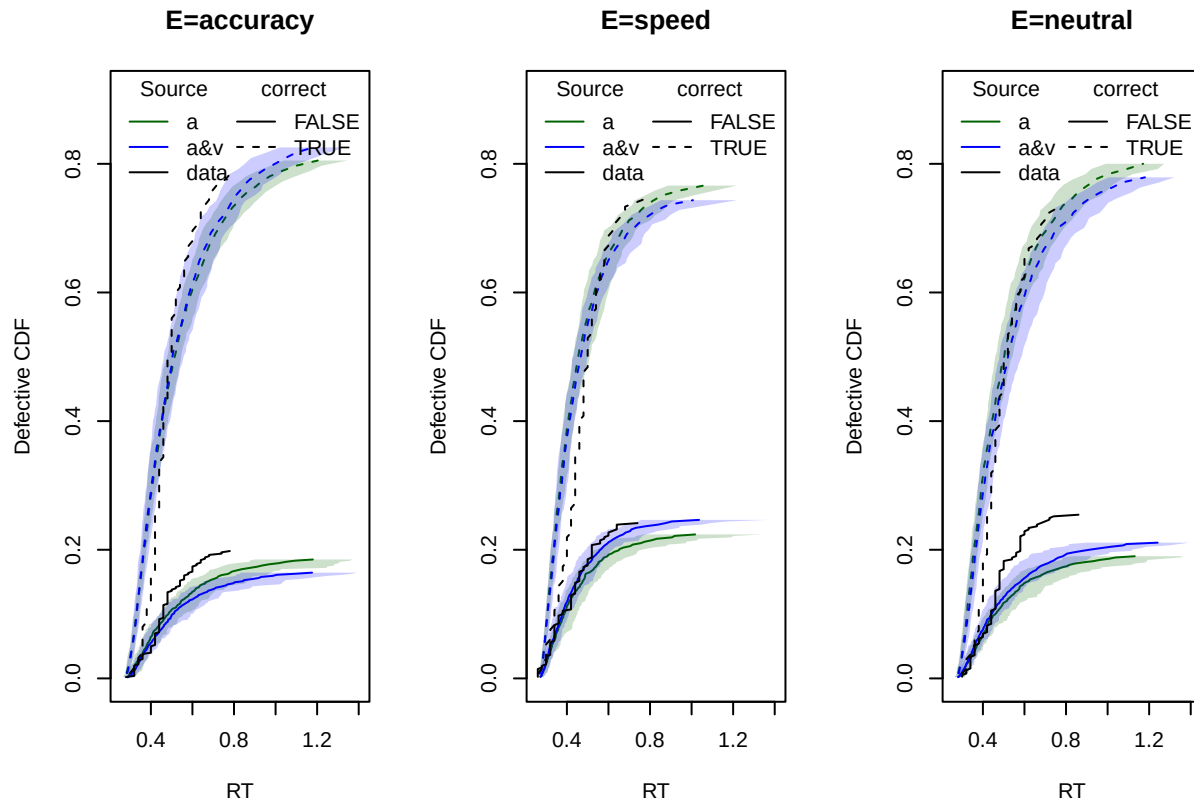
```
plot_cdf(DDMavE, pp_DDMavE, factors = "E", functions = c(correct = acc_fun),
  layout = c(1,3), defective_factor = "correct",
  legendpos = c("topright", "topleft"))
```



Comparing this model to the single subject data shows improved fit across subjects. The quantiles of the posterior predictive distribution are much tighter (barely visible on the plot), which is expected given the larger amount of data being fit.

We can also directly compare the fit of the new model to the old model for only the first subject (since the first model is only fit to the first subject):

```
plot_cdf(DDMaE, list(a = pp_DDMaE, 'a&v' = pp_DDMavE), factors = "E",
  functions = c(correct = acc_fun), layout = c(1,3),
  defective_factor = "correct",
  legendpos = c("topright", "topleft"), subject = 1)
```



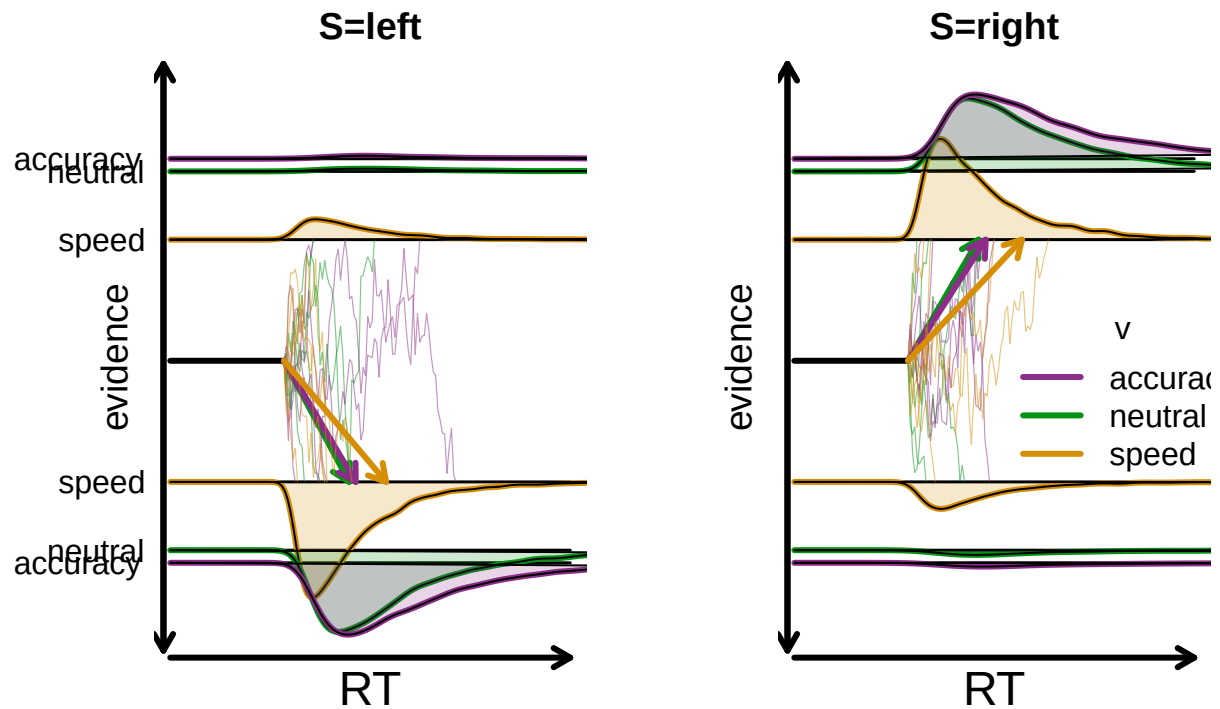
Note that despite the second model being more flexible, it is possible it provides a poorer fit to the data for this single subject due to shrinkage effects.

Model Comparison Now that we've examined model fit, we can test research questions.

For example, is there support for (conventional) threshold and (less conventional) drift rate differences across emphasis conditions at the population (μ) level?

We can plot the implied accumulation process of our estimated parameters using the `plot_design` function, but now with the `emc` object:

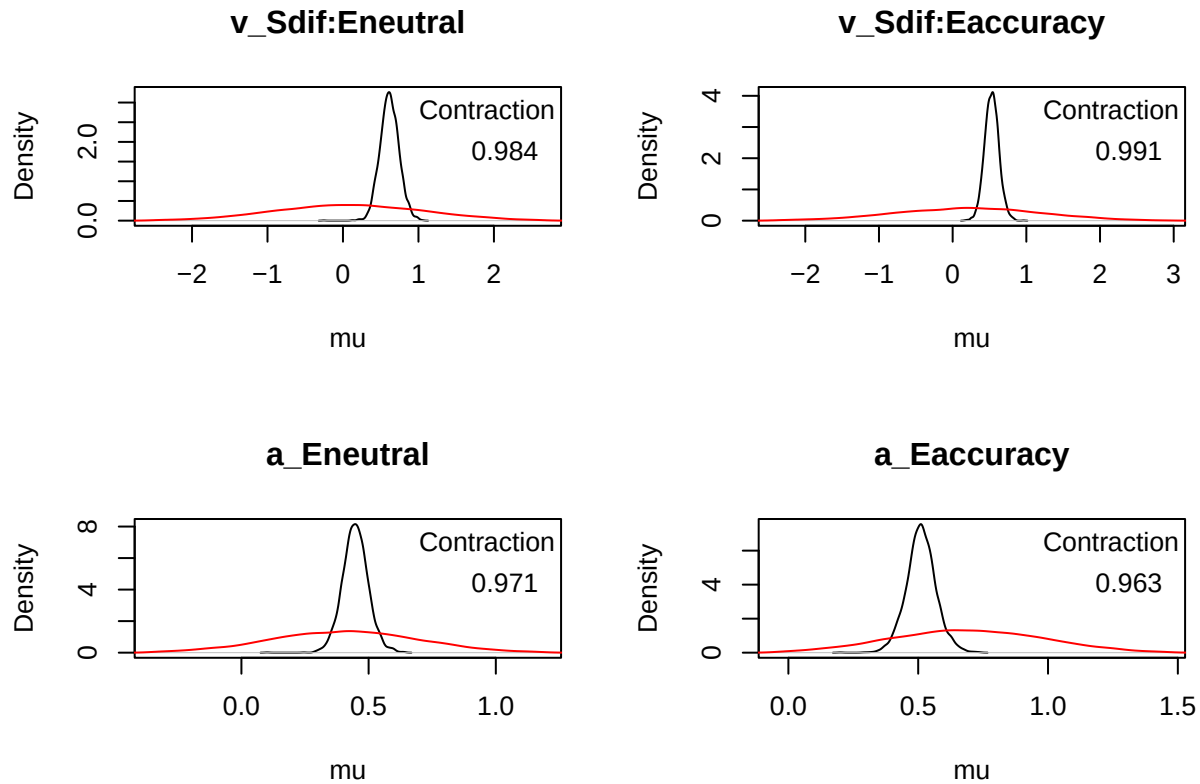
```
plot_design(DDMavE, factors = list(v = c("E", "S"), a = "E"), plot_factor = "S")
```

This suggests small differences in drift rates and thresholds between the neutral and accuracy conditions.

To get a better understanding of the effect, let's plot the posterior parameter densities for our parameters of interest:

```
plot_pars(DDMavE, selection = "mu",
          use_par = c("a_Eneutral", "a_Eaccuracy",
                    "v_Sdif:Eneutral", "v_Sdif:Eaccuracy"))
```



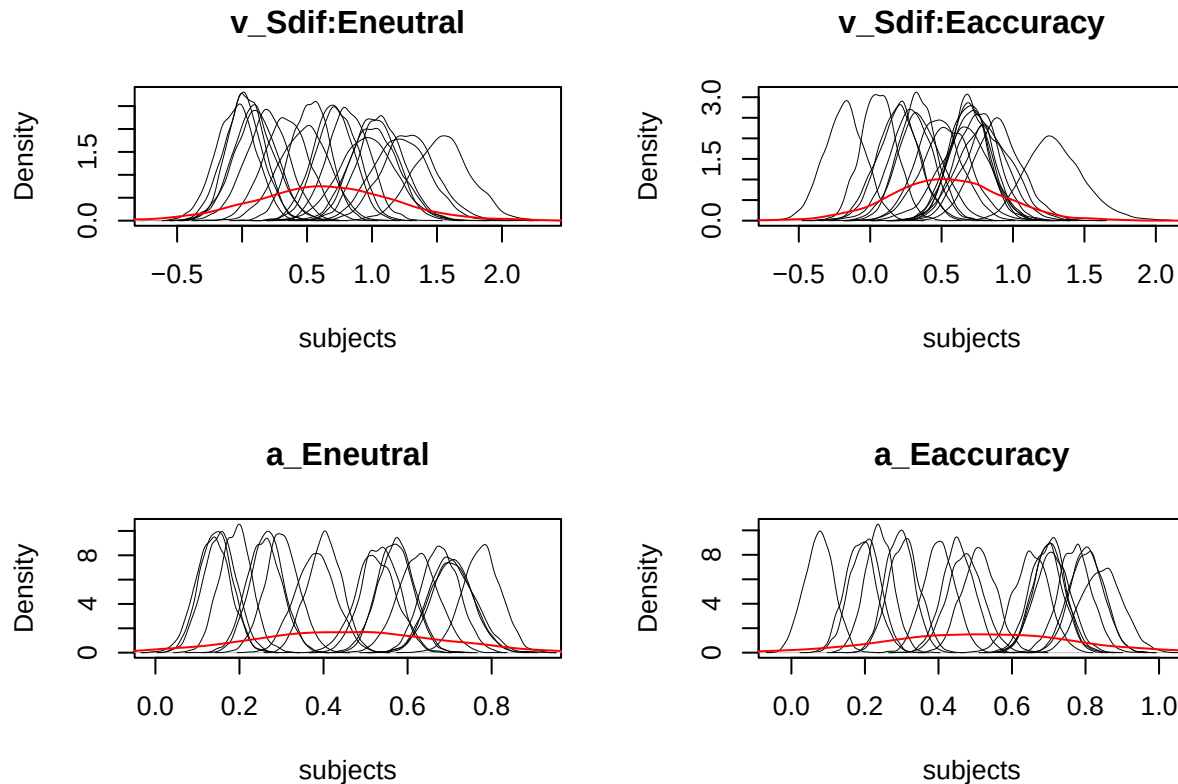
The following 95% credible intervals clearly show strong effects on both rates and thresholds, with similar magnitudes for neutral and accuracy effects:

```
credint(DDMavE, use_par = c("a_Eneutral", "a_Eaccuracy",
                             "v_Sdif:Eneutral", "v_Sdif:Eaccuracy"))
```

```
$mu
           2.5%  50% 97.5%
v_Sdif:Eneutral 0.381 0.615 0.868
v_Sdif:Eaccuracy 0.343 0.530 0.725
a_Eneutral      0.350 0.447 0.549
a_Eaccuracy     0.399 0.511 0.631
```

We can also visualize the individual differences in these effects by setting `all_subjects = TRUE`:

```
plot_pars(DDMavE, layout = c(2,2),
           use_par = c("a_Eneutral", "a_Eaccuracy",
                       "v_Sdif:Eneutral", "v_Sdif:Eaccuracy"),
           all_subjects = TRUE)
```



Here the group-level distribution (simulated from the estimated group-level mean and variance) is in red and the individual-level posteriors are in black.

Essentially the group-level distribution acts as the prior to the individual-level parameters. Notice that threshold effects are almost all above 0, whereas drift rate effects sometimes overlap zero, indicating that some participants do not pay more attention with greater accuracy emphasis.

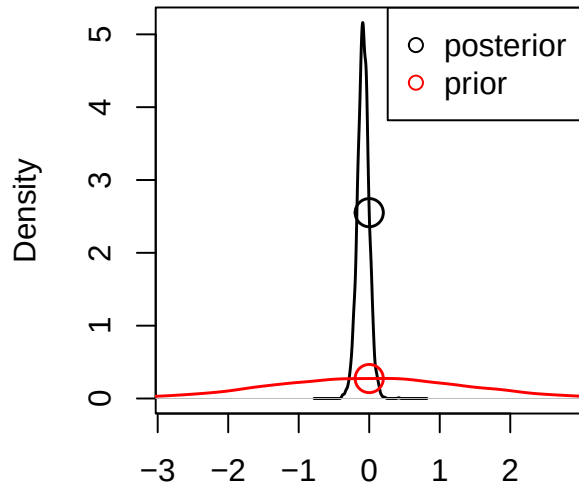
We can compute Bayes Factors for these parameter differences using the savage-dickey ratio, testing the group-level mean differences in drift rates and thresholds between the Neutral and Accuracy conditions. Values less than one favor the null hypothesis, and values greater than one favor the alternative hypothesis.

```
vdiff <- function(p)difff(p[c("v_Sdif:Eneutral","v_Sdif:Eaccuracy")])
adiff <- function(p)difff(p[c("a_Eneutral","a_Eaccuracy")])
par(mfrow=c(1,2))
hypothesis(DDMavE,fun=vdiff)
```

```
[1] 0.107395
```

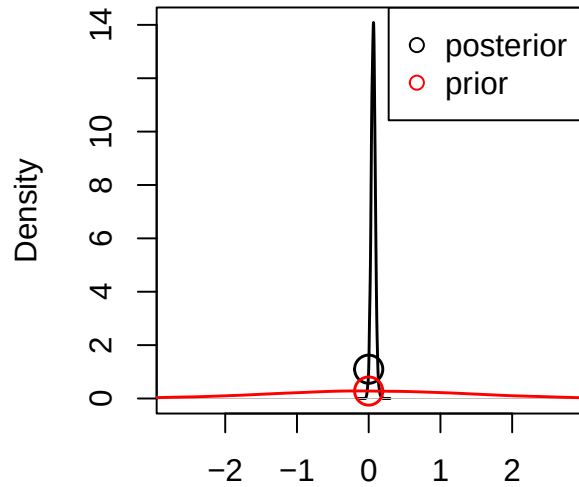
```
hypothesis(DDMavE,fun=adiff)
```

Prior and posterior density



N = 3000 Bandwidth = 0.01411

Prior and posterior density



N = 3000 Bandwidth = 0.005177

```
[1] 0.2552172
```

The plot shows the posterior (black) and prior (red) densities. The circles are plotted at the height of the densities at H_0 (in this case 0), since we're testing whether the differences between these parameters are larger than 0. If the prior is higher than the posterior at H_0 that means there's evidence for the effect, if lower evidence against it.

We find strong evidence against the drift rate difference between the neutral and accuracy condition, and more moderate evidence against the threshold difference. Thus suggesting that these conditions have similar effects on the drift rate and threshold settings. Similar conclusions are reached based on 95% credible intervals for these differences.

```
credible(DDMavE, x_fun=vdiff)
```

```
      fun mu
2.5% -0.25 NA
50%  -0.09  0
97.5% 0.08 NA
attr(,"less")
[1] 0.86
```

```
credible(DDMavE,x_fun=adiff)
```

```
      fun mu
2.5%  0.01 NA
50%   0.06  0
97.5% 0.12 NA
attr(,"less")
[1] 0.014
```

Linear Ballistic Accumulator Model (LBA)

Design Specification

We will now try to incorporate our previous findings in an LBA model. While we would normally do full model development for the DDM and LBA separately, this tutorial is just for illustrative purposes.

As the LBA is a race model it has one accumulator representing each possible response, in this case separate accumulators for the left and right response. EMC2 uses the levels of the “R” factor found in the data to construct a factor which denotes the accumulators “lR” (the latent response) with level names taken from Rlevels.

The lR factor allows for response bias, analogous to Z in the DDM. In race models, response bias can be modeled by allowing different thresholds (B) for different responses. Here we will use B~lR to allow for different thresholds for the accumulator corresponding to left and right stimuli (e.g., a bias to respond left occurs if the left threshold is lower than the right threshold).

For race models, you often need to provide a “matchfun” function that marks which accumulator matches the correct response. Typically, this function takes the latent response (lR) and returns TRUE when it matches the stimulus (S):

```
matchfun=function(data){data$S==data$lR}
```

matchfun is used to automatically create a latent match (lM) factor with levels “FALSE” (i.e., the stimulus does not match the accumulator) and “TRUE” (i.e., the stimulus does match the accumulator). This is added internally and can also be used in model formula, typically for parameters related to the rate of accumulation.

When working with lM, it’s often useful to design an “average and difference” contrast matrix. For binary responses, this has a simple form:

```
ADmat <- matrix(c(-1/2,1/2),ncol=1,dimnames=list(NULL,"d"))
ADmat
```

```
      d
[1,] -0.5
[2,]  0.5
```

When applied to drift rates (v~lM), the contrast parameter v_lMd represents the difference between correct and incorrect response drift rates. This “rate quality” parameter is analogous to the DDM drift rate. When accuracy exceeds chance, rate quality is positive.

The ADmat intercept (v) represents the average accumulation rate across accumulators, measuring both stimulus magnitude and decision urgency effects. Unlike the DDM, which represents only evidence differences between possible responses, this “rate quantity” parameter captures the overall tendency to respond. The DDM does not directly represent this tendency.

For simplicity, we omit rate bias, although it could be included by allowing v~S*lM.

We also allow the drift rate standard deviation to vary as a function of match (sv~lM), as has become standard in more recent applications of the LBA.

Just as with the DDM, we fix one parameter to make the model identifiable; in this case, as is common, we chose to fix drift rate standard deviation intercept (sv).

Given the findings of the hierarchical DDM that there’s no evidence for a threshold or drift rate difference between neutral and accuracy emphasis, we construct our model to estimate one shared E parameter for the neutral and accuracy condition. To achieve this, we use the functions argument.

```
E2 <- function(data) factor(data$E!="speed",labels=c("speed","nonspeed"))
design_LBABvE <- design(data = dat,model=LBA,matchfun=matchfun,
                        formula=list(v~lM*E2,sv~lM,B~E2+lR,A~1,t0~1),
                        contrasts=list(lM = ADmat),constants=c(sv=log(1)),
                        functions=list(E2=E2))
```

Sampled Parameters:

[1] "v"	"v_lMd"	"v_E2nonspeed"	"v_lMd:E2nonspeed"
[5] "sv_lMd"	"B"	"B_E2nonspeed"	"B_lRright"

```
[9] "A"          "t0"
```

Design Matrices:

```
$v
      1M      E2 v  v_lMd v_E2nonspeed v_lMd:E2nonspeed
TRUE   speed 1    0.5          0          0.0
FALSE  speed 1   -0.5          0          0.0
TRUE  nonspeed 1    0.5          1          0.5
FALSE  nonspeed 1   -0.5          1         -0.5
```

```
$sv
      1M sv sv_lMd
TRUE   1   0.5
FALSE  1  -0.5
```

```
$B
      E2      1R B B_E2nonspeed B_1Rright
speed left 1          0          0
speed right 1          0          1
nonspeed left 1          1          0
nonspeed right 1          1          1
```

```
$A
```

```
A
```

```
1
```

```
$t0
```

```
t0
```

```
1
```

Here the E2 function creates a new factor in the data, with levels speed and nonspeed.

Whereas the drift rate intercept in the DDM represents response bias, which is unlikely to be affected by emphasis instructions, in the LBA the intercept codes for urgency effects, which are more likely to be affected by emphasis instructions. We therefore allow the intercept to also vary by E, yielding a `V_E2nonspeed` parameter and a `v_lMd:E2nonspeed` parameter. The latter parameter represents the difference in accumulation rate between correct and incorrect responses under neutral/accuracy relative to the speed emphasis condition.

Let's examine how our parameters map to the design:

```
mapped_pars(design_LBABvE)
```

```
$v
      1M      E2
TRUE   speed    : v + 0.5 * v_lMd
FALSE  speed    : v - 0.5 * v_lMd
TRUE   nonspeed : v + 0.5 * v_lMd + v_E2nonspeed + 0.5 * v_lMd:E2nonspeed
FALSE  nonspeed : v - 0.5 * v_lMd + v_E2nonspeed - 0.5 * v_lMd:E2nonspeed
```

```
$sv
```

```
1M
```

```
TRUE   : exp(sv + 0.5 * sv_lMd)
```

```
FALSE  : exp(sv - 0.5 * sv_lMd)
```

```
$B
```

```
E2      1R
```

```

speed    left   : exp(B)
speed    right  : exp(B + B_lRight)
nonspeed left   : exp(B + B_E2nonspeed)
nonspeed right : exp(B + B_E2nonspeed + B_lRight)

```

Prior Specification

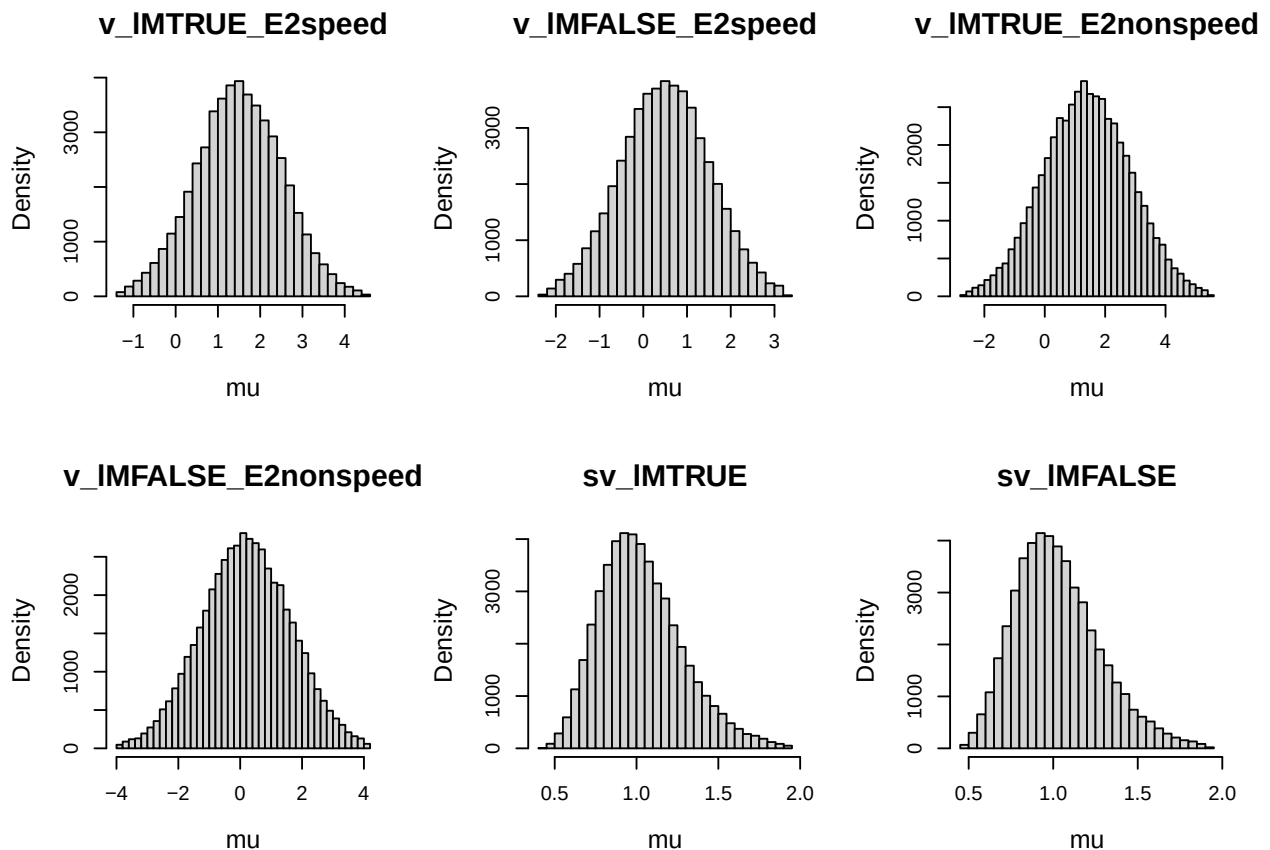
We now create a prior. Note that all parameters except v are estimated on a log scale (see ?LBA). For the LBA, t_0 is typically smaller than for the DDM, so we set it to 0.2. We further set increasing thresholds and urgency for higher response caution conditions. As we have no response bias expectation, B_lRight is set to zero.

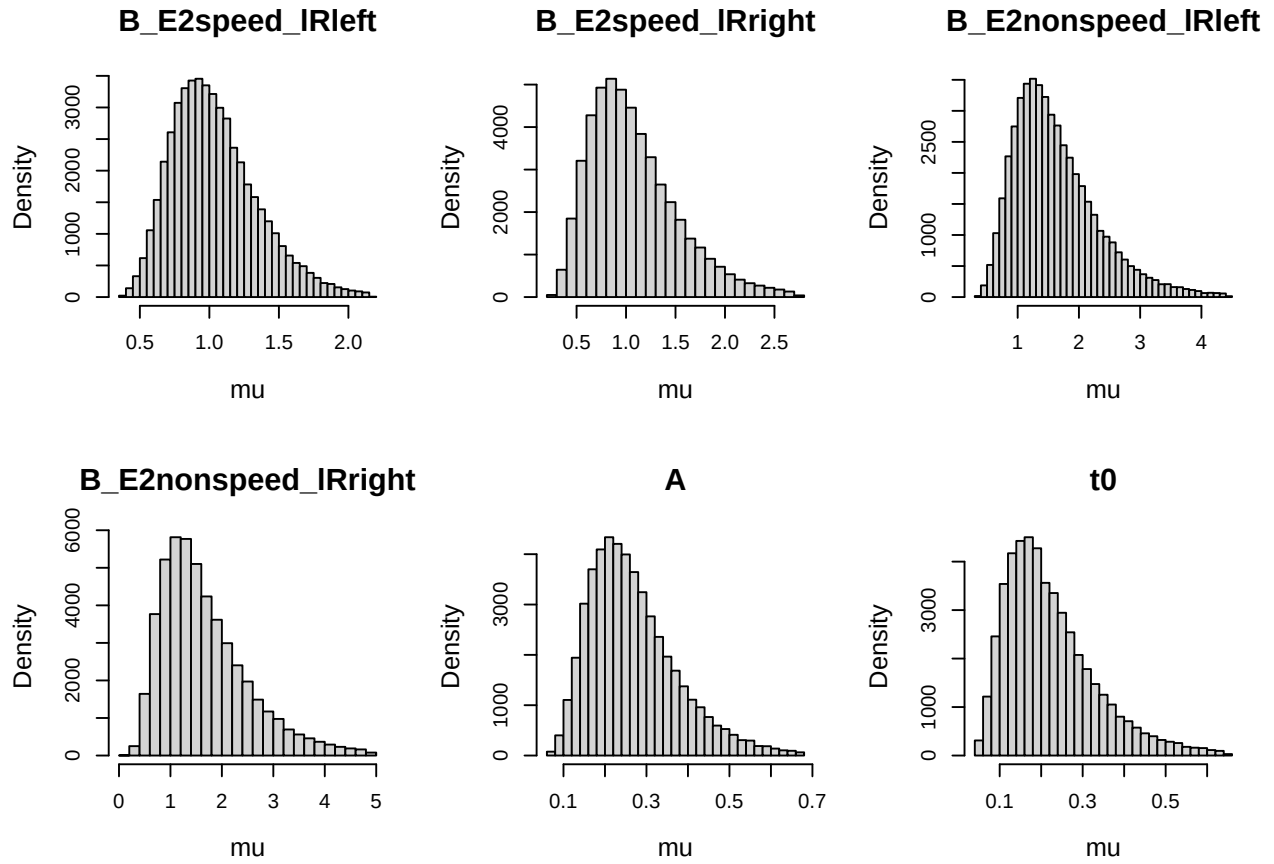
```

mu_mean <- c(v=1, v_E2nonspeed = -.2, v_lMd=1, "v_lMd:E2nonspeed"=.2,
             sv_lMd=log(1),B=log(1), B_E2nonspeed = log(1.5), B_lRight=0,
             A=log(0.25), t0=log(.2))
mu_sd <- c(v=1, v_lMd=0.5, "v_lMd:E2nonspeed"=0.5,
           sv_lMd=.5,B=0.3, B_E2nonspeed=0.3,B_lRight=0.3, A=0.4, t0=.5)

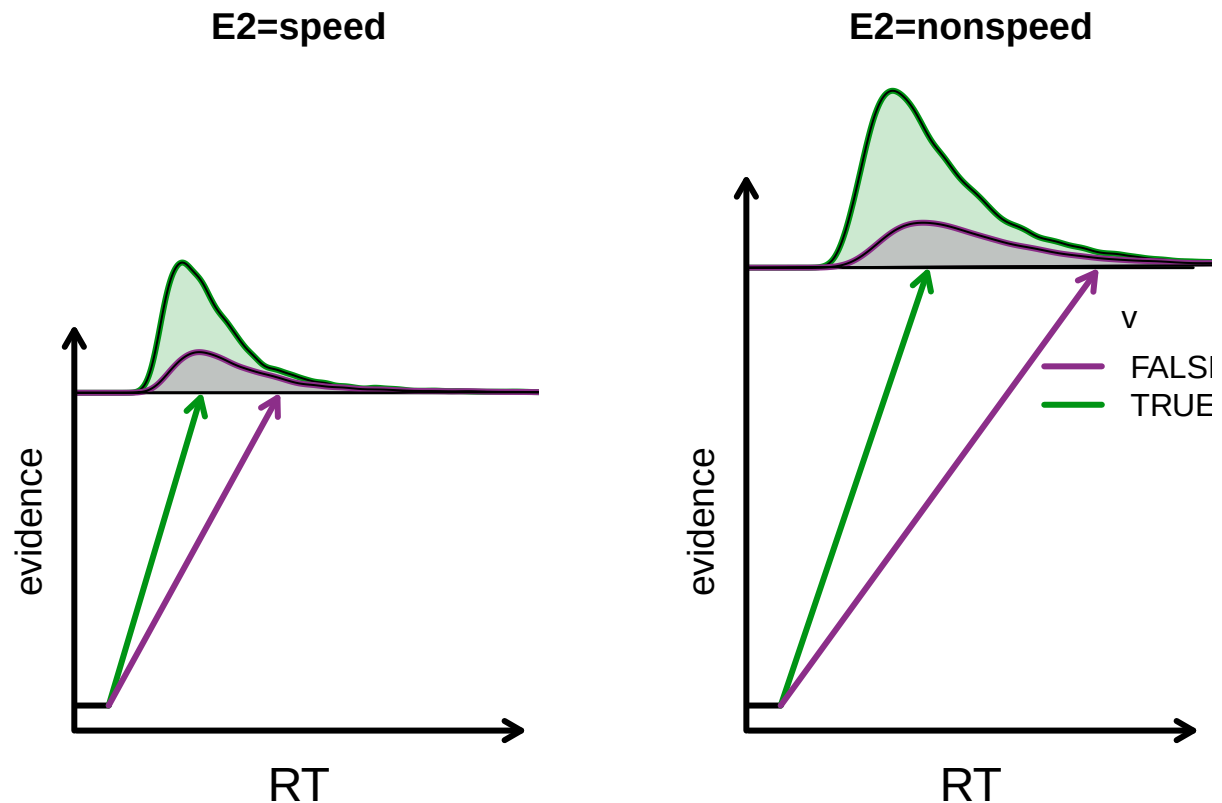
prior_LBABvE <- prior(design_LBABvE, type = 'standard',mu_mean=mu_mean,
                     mu_sd=mu_sd)
plot(prior_LBABvE, layout = c(2,3))

```





```
plot_design(prior_LBABvE, factors = list(v = c("E2", "1M"), B = "E2"),
  plot_factor = "E2")
```

Model Estimation

Now we can fit the model:

```
LBABvE <- make_emc(dat,design_LBABvE, prior=prior_LBABvE)
LBABvE <- fit(LBABvE, cores_per_chain = 4, fileName="tmp.RData")
save(LBABvE,file="LBABvE.RData")
```

The model shows good convergence and generally good efficiency:

```
check(LBABvE)
```

Iterations:

	preburn	burn	adapt	sample
[1,]	0	0	0	1000
[2,]	0	0	0	1000
[3,]	0	0	0	1000

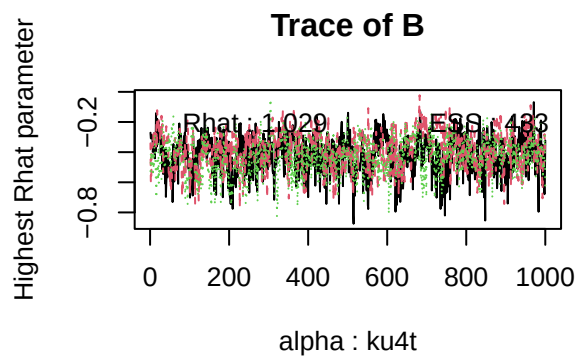
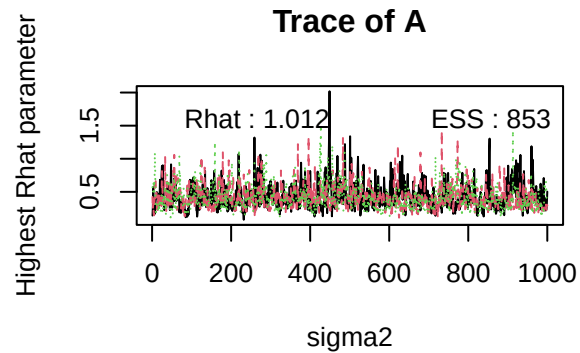
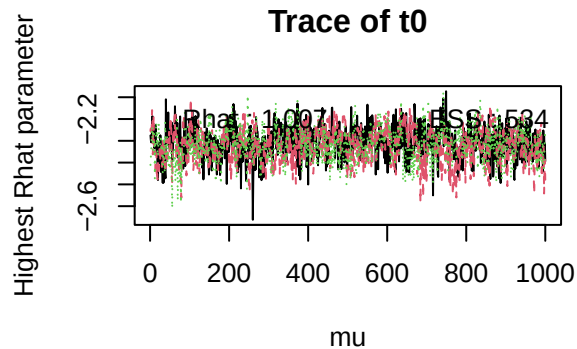
	mu				sigma2			
	v	v_lMd	v_E2nonspeed	v_lMd:E2nonspeed	sv_lMd	B	B_E2nonspeed	
Rhat	1.004	1	1.004	1.002	1	1.004	1.004	
ESS	1628.000	2261	1682.000	2031.000	2728	771.000	726.000	
	B_lRright	A	t0					
Rhat	1.001	1.006	1.007					
ESS	2298.000	1213.000	534.000					

	sigma2					
	v	v_lMd	v_E2nonspeed	v_lMd:E2nonspeed	sv_lMd	B
Rhat	1.004	1.006	1.003	1.006	1.002	1.002

ESS	1087.000	1222.000	1455.000	1302.000	1242.000	940.000
	B_E2nonspeed	B_lRright	A	t0		
Rhat	1.002	1.002	1.012	1.009		
ESS	892.000	1666.000	853.000	725.000		

alpha highest Rhat : ku4t

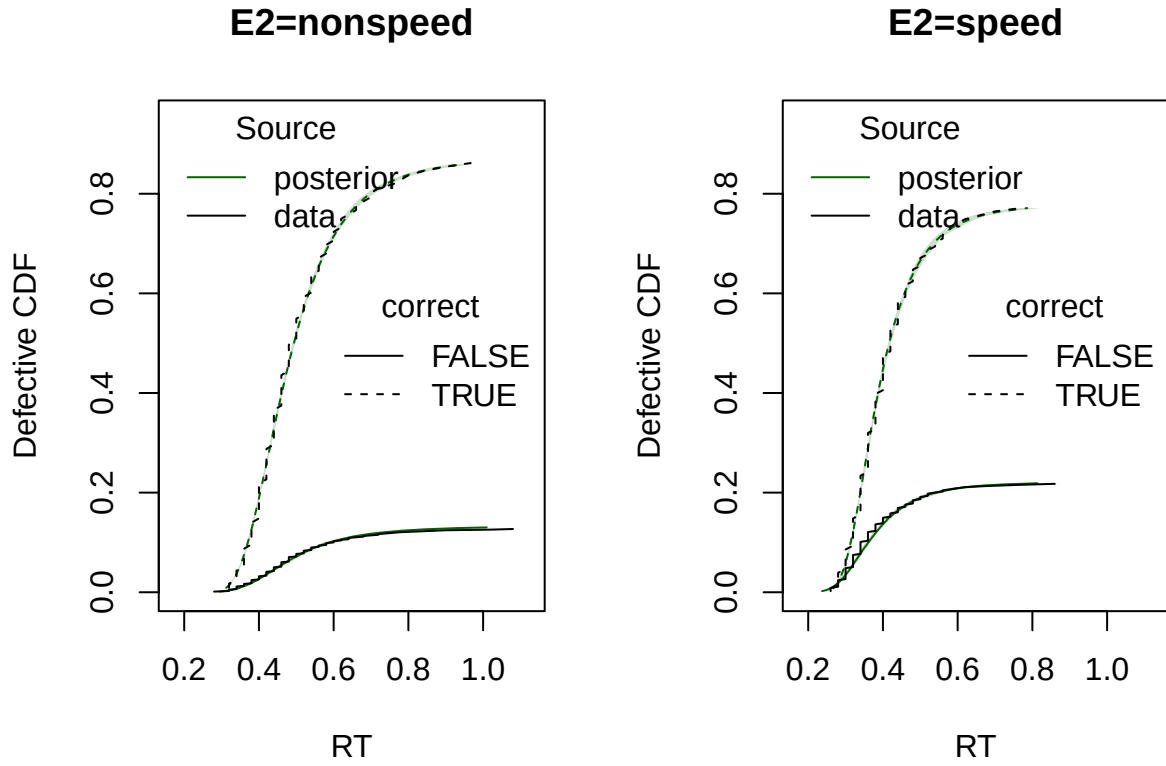
	v	v_lMd	v_E2nonspeed	v_lMd:E2nonspeed	sv_lMd	B
Rhat	1.016	1.002	1.01	1.001	1.001	1.029
ESS	593.000	2204.000	1001.00	2230.000	2371.000	433.000
	B_E2nonspeed	B_lRright	A	t0		
Rhat	1.017	1.004	1.017	1.024		
ESS	709.000	2121.000	839.000	560.000		



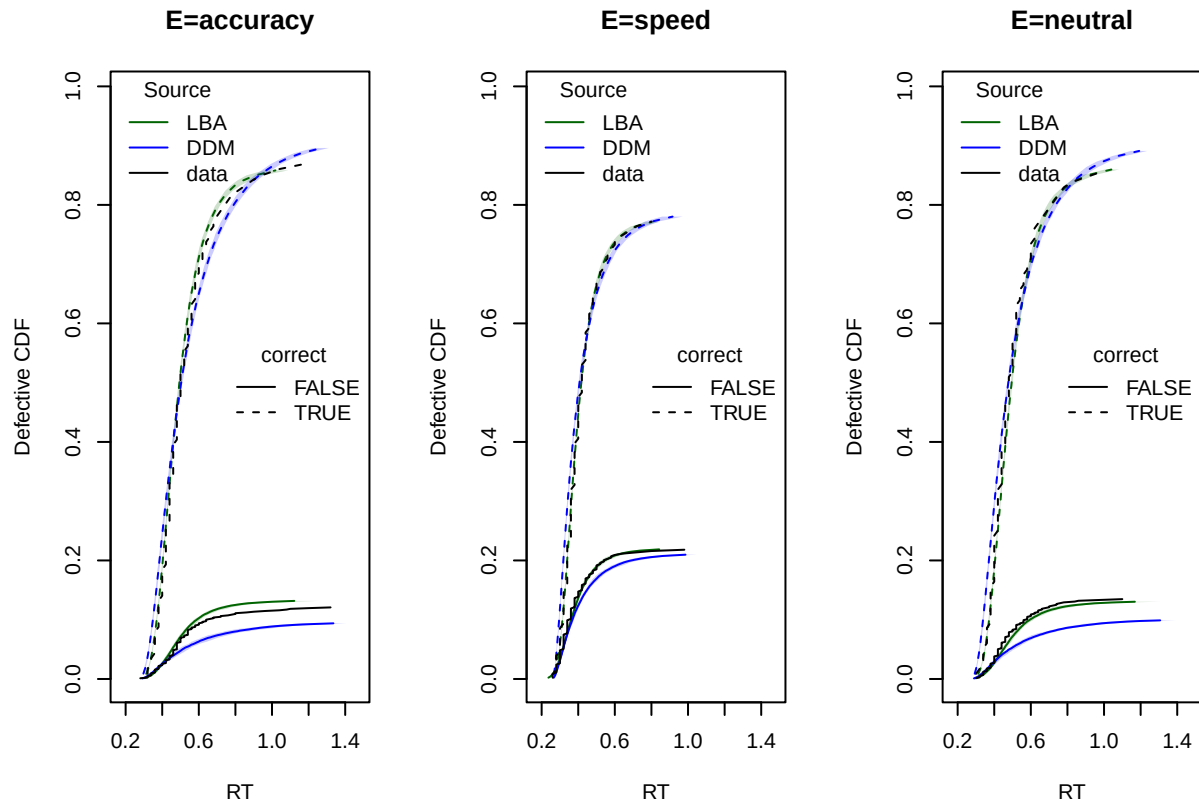
Model Inference

Model Fit We generate posterior predictions for the LBA model to see how well the model fits the data:

```
pp_LBABvE <- predict(LBABvE, n_cores = 12)
plot_cdf(LBABvE, pp_LBABvE, factors = "E2", functions = c(correct = acc_fun),
         layout= c(1,2), defective_factor = "correct",
         legendpos = c("right", "topleft"))
```



We can also directly compare the fit of the LBA model to the DDM model:

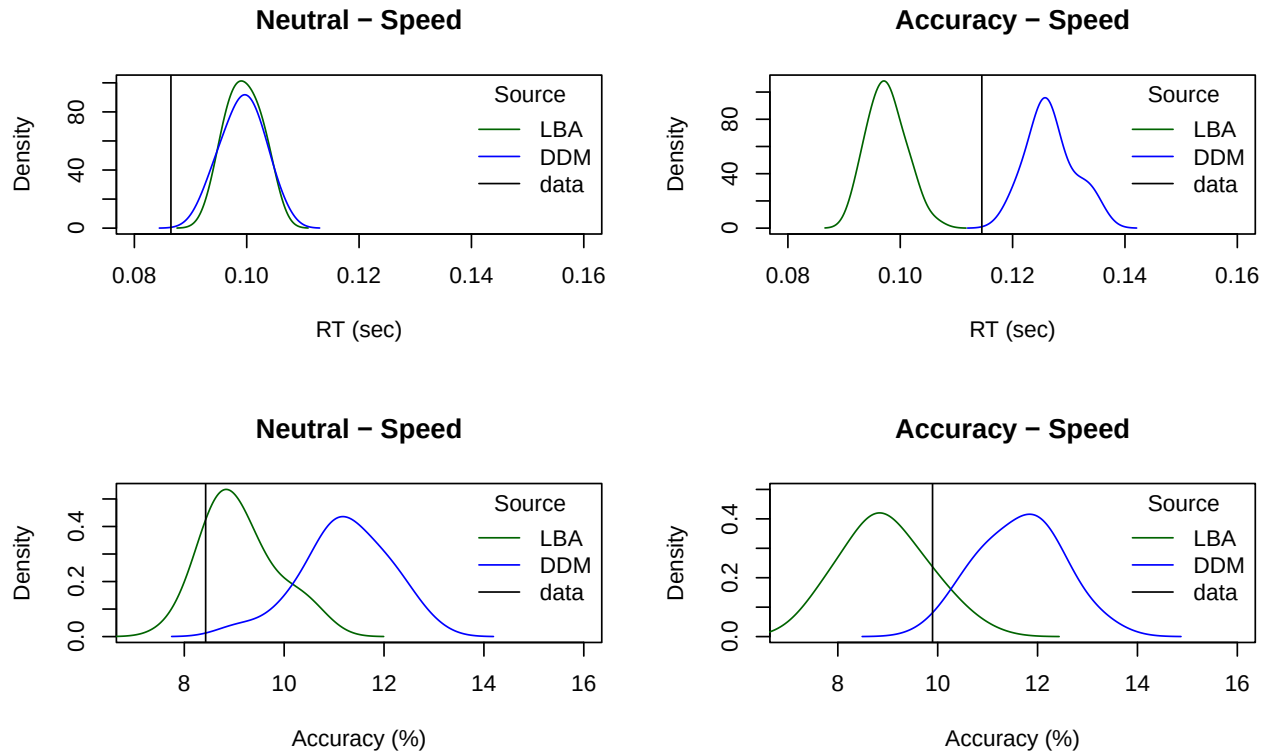


The models show different patterns of misfit. The DDM underestimates the number of error responses, especially in the neutral condition. In contrast, the LBA model slightly overestimates error responses in the accuracy condition. Note that since we specified that the LBA model should only estimate one E parameter

for the neutral and accuracy condition, the LBA posterior predictives do not show a difference between these conditions. Although the DDM includes that flexibility, it does not include separate urgency effects for the neutral and accuracy condition (as it does not include urgency effects).

We can also compare arbitrary descriptives on the real data to the predictives of both models. For example differences in response time and error rates between the emphasis conditions:

```
drt_ns <- function(data){
  all <- tapply(data$rt,data$E,mean)
  out <- all['neutral'] - all['speed']
  names(out) <- "Neutral - Speed"
  return(out)
}
drt_as <- function(data){
  all <- tapply(data$rt,data$E,mean)
  out <- all['accuracy'] - all['speed']
  names(out) <- "Accuracy - Speed"
  return(out)
}
derr_ns <- function(data){
  data$correct <- data$S == data$R
  all <- tapply(data$correct,data$E,mean)*100
  out <- all['neutral'] - all['speed']
  names(out) <- "Neutral - Speed"
  return(out)
}
derr_as <- function(data){
  data$correct <- data$S == data$R
  all <- tapply(data$correct,data$E,mean)*100
  out <- all['accuracy'] - all['speed']
  names(out) <- "Accuracy - Speed"
  return(out)
}
par(mfrow = c(2,2))
plot_stat(LBABvE, list(LBA = pp_LBABvE, DDM = pp_DDMavE), stat_fun = drt_ns,
  legendpos = c('topleft', 'topright'), main = "Neutral - Speed",
  layout=NULL,xlim=c(.08,.16),adjust=1.5,xlab = "RT (sec)")
plot_stat(LBABvE, list(LBA = pp_LBABvE, DDM = pp_DDMavE), stat_fun = drt_as,
  legendpos = c('topleft', 'topright'), main = "Accuracy - Speed",
  layout=NULL,xlim=c(.08,.16),adjust=1.5,xlab = "RT (sec)")
plot_stat(LBABvE, list(LBA = pp_LBABvE, DDM = pp_DDMavE), stat_fun = derr_ns,
  legendpos = c('topleft', 'topright'), main = "Neutral - Speed",
  layout=NULL,xlim=c(7,16),adjust=1.5, xlab = "Accuracy (%)")
plot_stat(LBABvE, list(LBA = pp_LBABvE, DDM = pp_DDMavE), stat_fun = derr_as,
  legendpos = c('topleft', 'topright'), main = "Accuracy - Speed",
  layout=NULL,xlim=c(7,16),adjust=1.5, xlab="Accuracy (%)")
```



Here it is clear that the LBA model fits better in terms of accuracy differences, whereas both models have similar quality accounts of RT differences, but with opposite misfit for the accuracy - speed difference.

Model Comparison Although visual fit is helpful in assessing absolute model fit, statistical differences between the models can be better tested using Bayes Factors. These take into account how well the model fits all aspects of the entire data set, while penalizing for model complexity.

EMC2 estimates Bayes Factors between sets of samples using the Warp-III bridge sampling method. To assess numerical stability see `both_splits` in `run_bridge_sampling`.

```
compare(list(LBA = LBABvE, DDM = DDMavE))
```

	MD	wMD	DIC	wDIC	BPIC	wBPIC	EffectiveN	meanD	Dmean	minD
LBA	-15673	1	-16955	1	-16743	1	212	-17167	-17326	-17379
DDM	-11103	0	-13233	0	-12986	0	247	-13480	-13661	-13727

Here the MD (Marginal Deviance; used for Bayes Factors), DIC (Deviance Information Criterion) and BPIC (Bayesian Predictive Information Criterion) are measures of model fit. The lower the better. The weights (wX) are the probabilities of the models, the higher the better.

The model comparison indicated that the LBA model provides a better account of the data. We can now use our winning model to do some further hypothesis testing. For example, how much evidence do we have for threshold, quality and quantity differences between the speed and non-speed conditions?

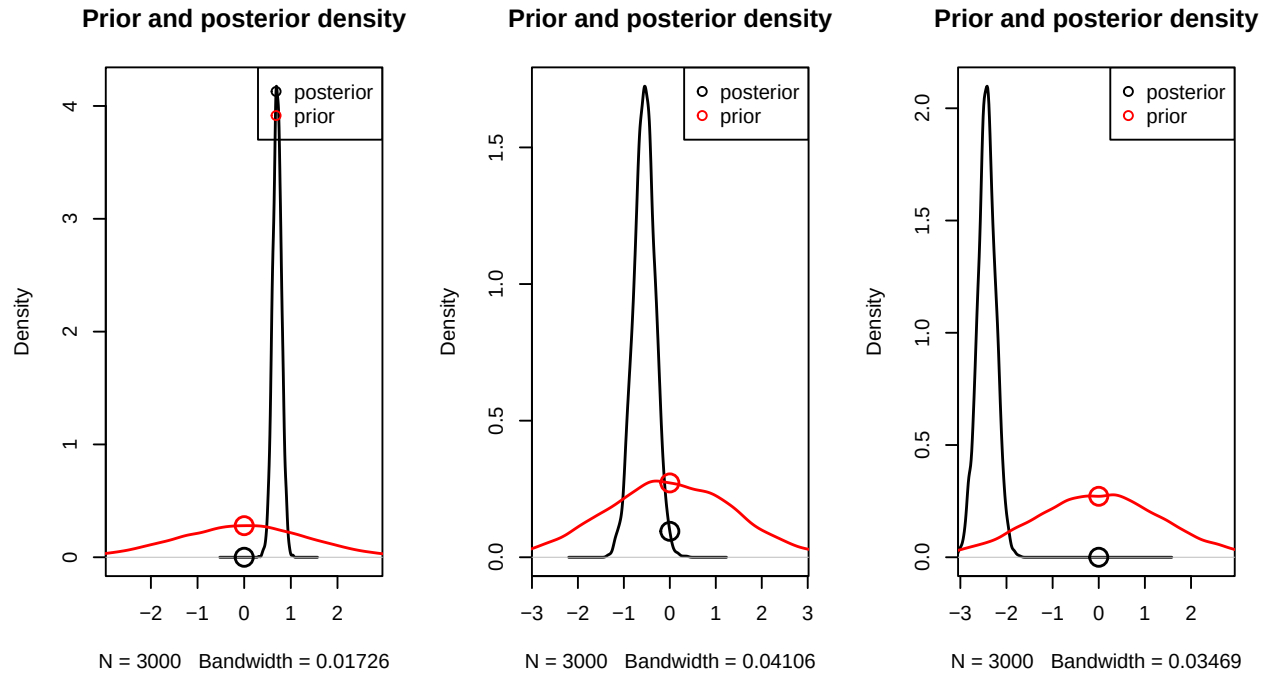
```
b_diff <- function(p)diff(p[c("B", "B_E2nonspeed")])
qual_diff <- function(p)diff(p[c("v_lMd", "v_lMd:E2nonspeed")])
quant_diff <- function(p)diff(p[c("v", "v_E2nonspeed")])
par(mfrow=c(1,3))
hypothesis(LBABvE, fun=b_diff)
```

```
[1] 2.073146e+15
```

```
hypothesis(LBABvE,fun=qual_diff)
```

```
[1] 2.864957
```

```
hypothesis(LBABvE,fun=quant_diff)
```



```
[1] 5.805268e+15
```

The results show very strong evidence for threshold and urgency differences but only moderate evidence for a quality difference.

To see the direction of these differences:

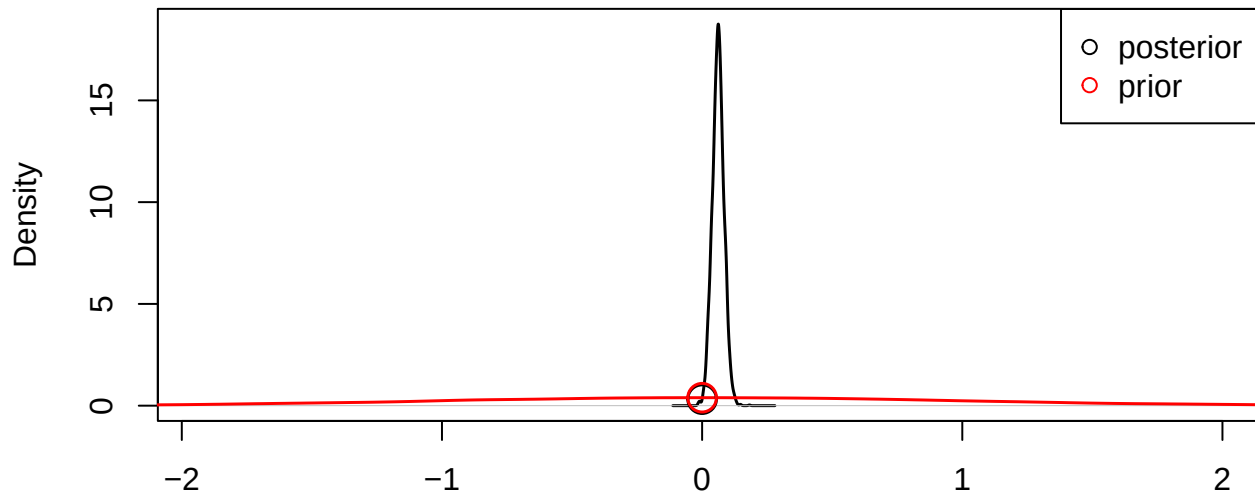
```
credint(LBABvE, use_par = c("v_E2nonspeed", "v_lMd:E2nonspeed", "B_E2nonspeed"))
```

```
$mu
      2.5%   50%  97.5%
v_E2nonspeed -0.840 -0.561 -0.244
v_lMd:E2nonspeed 0.662 1.068 1.417
B_E2nonspeed 0.198 0.263 0.328
```

Lastly, we can test for evidence of response bias:

```
hypothesis(LBABvE,"B_lRright")
```

Prior and posterior density



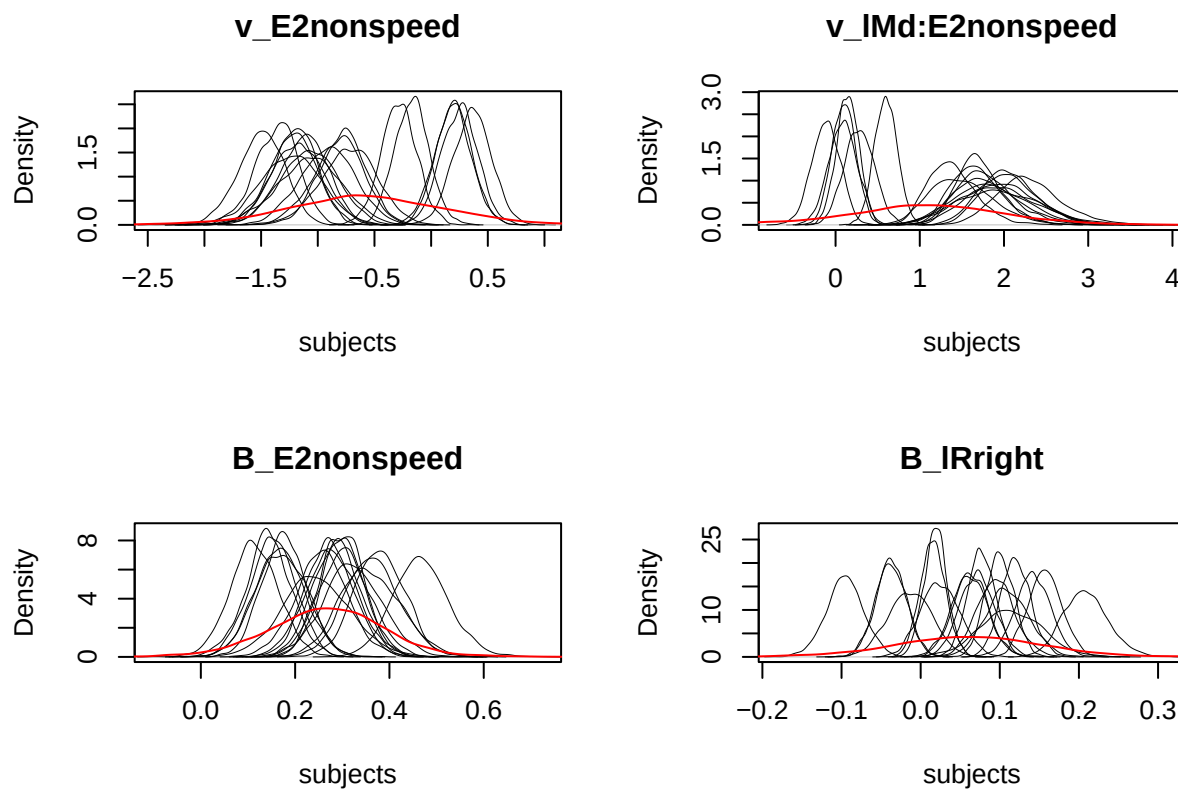
N = 3000 Bandwidth = 0.003962

[1] 1.227889

Evidence for bias is equivocal bias, with a slight tendency favoring the left response (i.e., a higher threshold for right).

We finish by inspecting the individual differences among the parameters of interest:

```
plot_pars(LBABvE, all_subjects = TRUE, use_par = c("B_lRright", "B_E2nonspeed",
  "v_E2nonspeed", "v_lMd:E2nonspeed"))
```



Everybody seems to have an E threshold effect, quality and quantity effects are more variable, and biases even more so.

This tutorial is not intended as a complete model comparison. The DDM could have potentially been more competitive in terms of complexity penalized model selection with similar parameter constraints on the emphasis effects to the LBA, and improved in absolute fit by allowing for additional between-trial variability in the parameters (st0 and SZ). Similarly, for the LBA, the misfit to RT differences between the speed condition and the other two conditions could be addressed by allowing rates and/or thresholds to differ between the accuracy and neutral conditions.